

Engenharia de Software – PUC Campinas

Estrutura e Recuperação de Dados I

Prof. Dr. Paulo Cesar de Alvarenga Lucci

Filas (queue)

Filas

○ INTRODUÇÃO

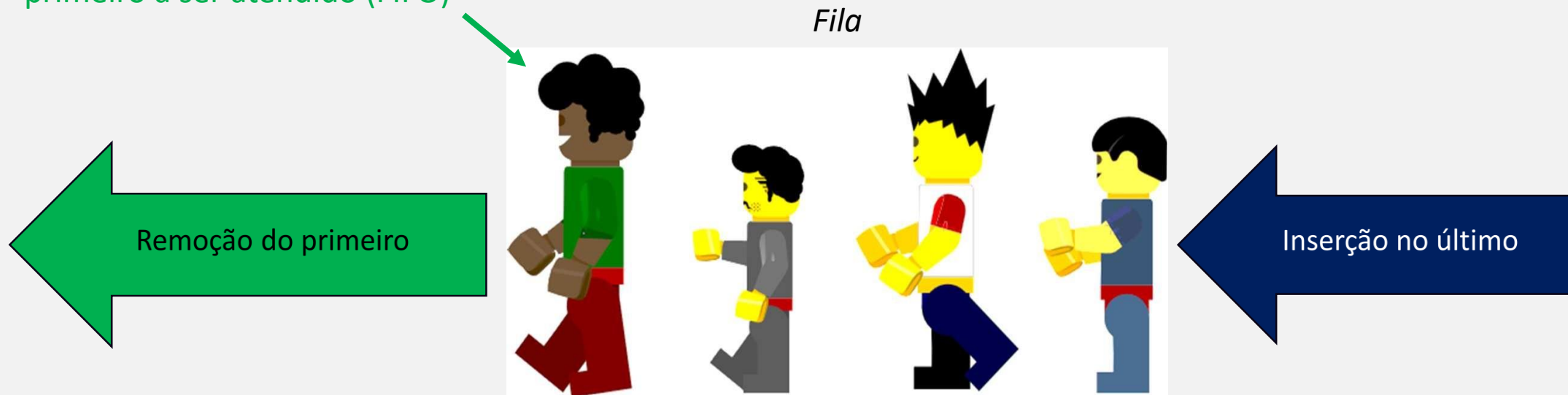
- Já estudamos listas encadeadas, em que a ideia principal é nós que guardam dados e um ponteiro para o próximo nó.
- Existem **comportamentos específicos** de listas que as tornam um **outro tipo**, só para que seja possível estabelecer um **protocolo** de **comportamento**.
- Uma analogia é de pessoas que correm (corredores).
 - Podemos derivar desse conceito **corredores** que têm como objetivo:
 - Colocar uma bola com os pés dentro de um gol (**jogador de futebol**).
 - Colocar uma bola com as mãos dentro de um aro preso a uma tabela (**jogador de basquete**).
 - Etc.
- Diante dessa ideia, falaremos a respeito de **filas**, que é uma **lista encadeada** com **comportamentos específicos**.

Filas

○ O QUE SÃO FILAS?

- É uma **lista encadeada** especializada que atua em uma estratégia muito bem definida chamada **FIFO**.
 - **FIFO** – **F**irst **I**n, **F**irst **O**ut ou seja: “o primeiro que entra é o primeiro que sai”.
- Percebam que não é à toa que se chama “fila” pois é o **mesmo princípio da fila de atendimento da vida real**, que obedece o **critério de chegada**.

Primeiro a chegar será o primeiro a ser atendido (FIFO)



Filas

○ INSERT E REMOVE

- Pelo fato de a fila ter o comportamento FIFO, os métodos de inserir e remover já trazem em si que:
 - **insert** = insert_back.
 - **remove** = remove_front.
- Por isso em implementações de filas, é comum ter as funções **insert** e **remove** sem os sufixos **_back** e **_front**.
 - Cuidado com alguns compiladores que têm as sintaxes “insert” e “remove” reservadas.

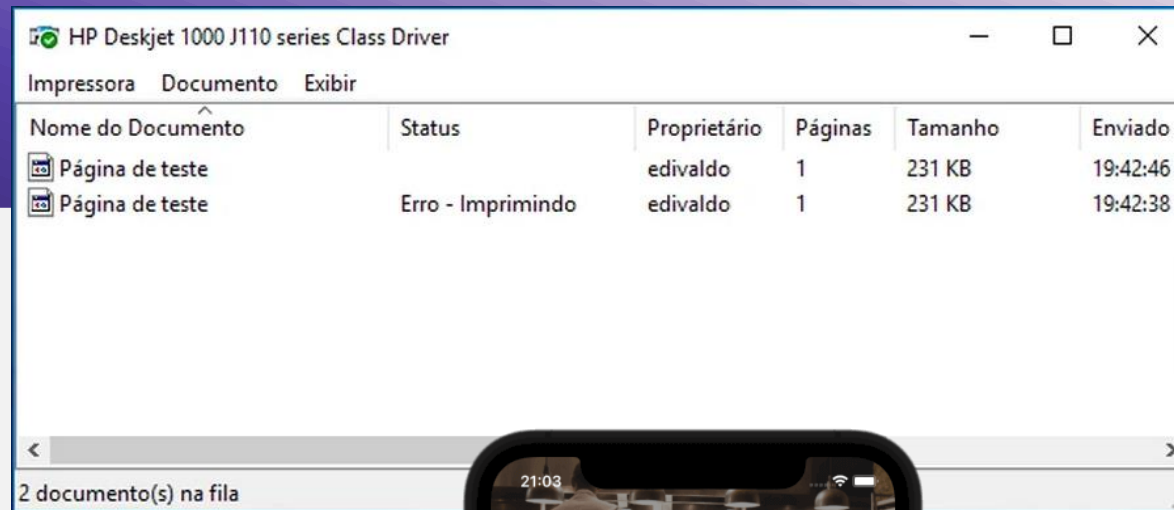
Fila



Filas

○ APLICAÇÕES

- Impressoras (ordem de documentos).
- Processos em sistemas operacionais.
- Chamadas telefônicas.
- Gerenciamento de filas em bancos, farmácias, hospitais etc.

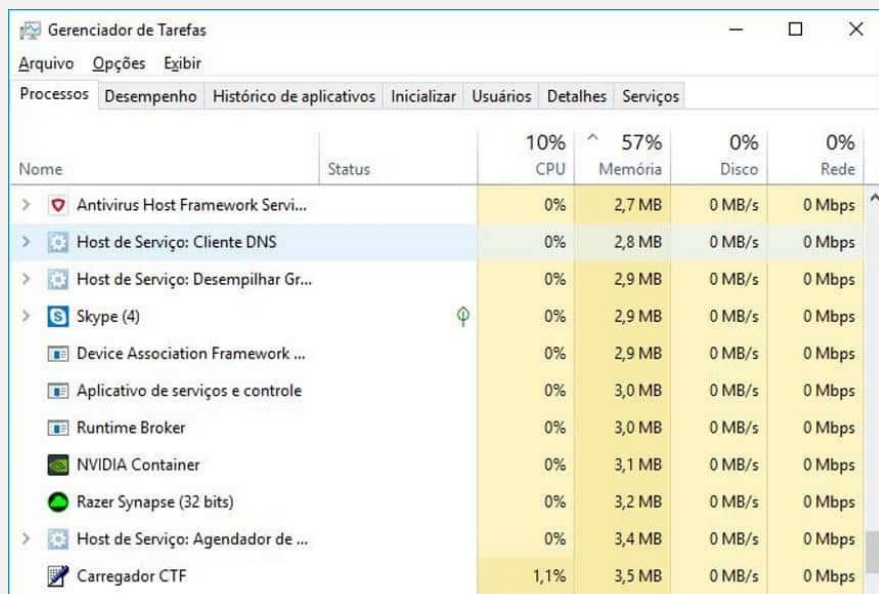


HP Deskjet 1000 J110 series Class Driver

Impressora Documento Exibir

Nome do Documento	Status	Proprietário	Páginas	Tamanho	Enviado
Página de teste		edivaldo	1	231 KB	19:42:46
Página de teste	Erro - Imprimindo	edivaldo	1	231 KB	19:42:38

2 documento(s) na fila



Gerenciador de Tarefas

Arquivo Opções Exibir

Processos Desempenho Histórico de aplicativos Inicializar Usuários Detalhes Serviços

Nome	Status	10% CPU	57% Memória	0% Disco	0% Rede
> Antivirus Host Framework Servi...		0%	2,7 MB	0 MB/s	0 Mbps
> Host de Serviço: Cliente DNS		0%	2,8 MB	0 MB/s	0 Mbps
> Host de Serviço: Desempenhar Gr...		0%	2,9 MB	0 MB/s	0 Mbps
> Skype (4)		0%	2,9 MB	0 MB/s	0 Mbps
Device Association Framework ...		0%	2,9 MB	0 MB/s	0 Mbps
Aplicativo de serviços e controle		0%	3,0 MB	0 MB/s	0 Mbps
Runtime Broker		0%	3,0 MB	0 MB/s	0 Mbps
NVIDIA Container		0%	3,1 MB	0 MB/s	0 Mbps
Razer Synapse (32 bits)		0%	3,2 MB	0 MB/s	0 Mbps
> Host de Serviço: Agendador de ...		0%	3,4 MB	0 MB/s	0 Mbps
Carregador CTF		1,1%	3,5 MB	0 MB/s	0 Mbps

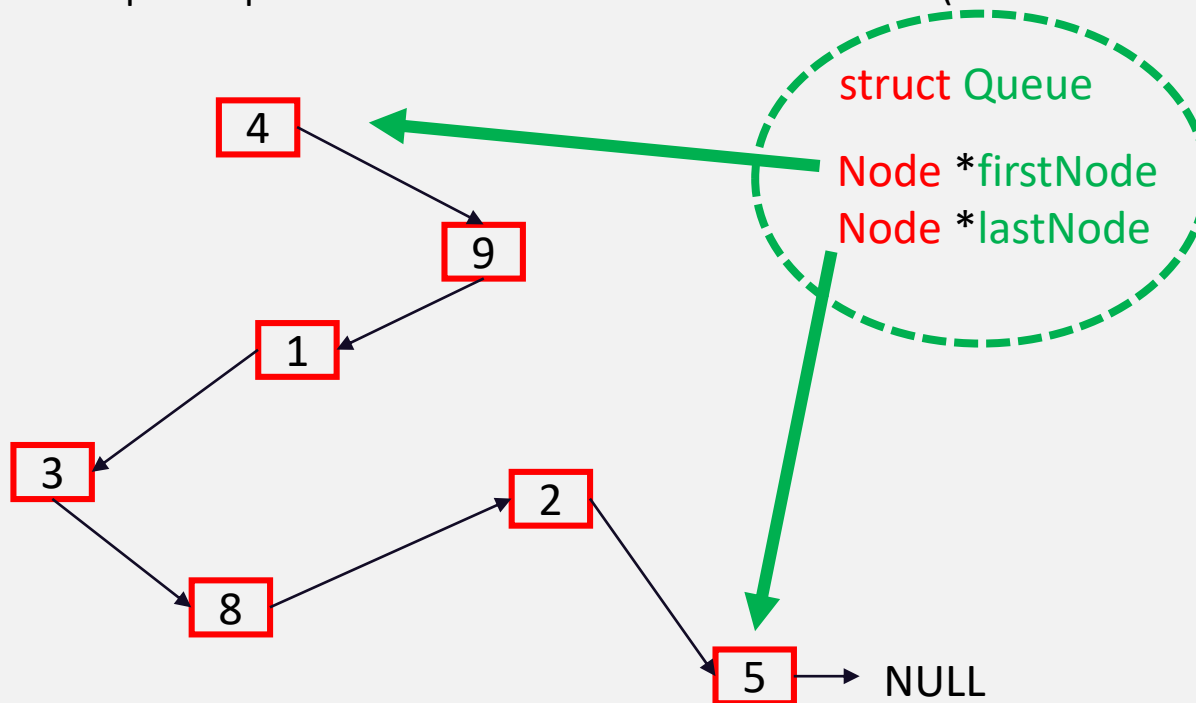


ESSAS APLICAÇÕES UTILIZAM ESTRUTURAS DE FILAS!!!

Filas

○ ESTRUTURA DE FILAS

- Vimos que as listas encadeadas são nós que guardam dados e um ponteiro para o próximo nó.
- Como a **inserção** numa fila sempre será no **final** da lista encadeada associada, temos duas opções:
 1. Percorrer toda a lista até seu final para, finalmente, inserir o novo nó (**pouco eficiente**). ❌
 2. Guardar um ponteiro para o primeiro e último nó da lista encadeada (**muito eficiente**). ✅



Filas

○ IMPLEMENTAÇÃO DE FILA

main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Node structure
5 typedef struct Node {
6     int value;
7     struct Node* next;
8 } Node;
9
10 // Queue structure
11 typedef struct Queue {
12     Node* first;
13     Node* last;
14 } Queue;
15
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
typedef struct Node {
    int value;
    struct Node* next;
} Node;

// Queue structure
typedef struct Queue {
    Node* first;
    Node* last;
} Queue;

// Create a new node
Node* create_node(int value) {
    Node* new_node = (Node*) malloc(sizeof(Node));
    new_node->value = value;
    new_node->next = NULL;
    return new_node;
}

// Insert (enqueue) - insere no fim da fila
Queue* insert_node(Queue* q, int value) {
    Node* new_node = create_node(value);

    if (q->last == NULL) {
        // Fila estava vazia
        q->first = new_node;
        q->last = new_node;
    } else {
        q->last->next = new_node;
        q->last = new_node;
    }

    return q;
}

// Remove (dequeue) - remove do inicio da fila
Queue* remove_node(Queue* q) {
    if (q->first == NULL) return q; // Fila vazia

    Node* temp = q->first;
    q->first = q->first->next;

    if (q->first == NULL) {
        // Fila ficou vazia
        q->last = NULL;
    }

    free(temp);
    return q;
}

// Check if queue is empty
int is_empty(Queue* q) {
    return (q->first == NULL);
}

// Free the queue
void free_queue(Queue* q) {
    Node* current = q->first;
    while (current != NULL) {
        Node* temp = current;
        current = current->next;
        free(temp);
    }
    free(q);
}

// Print the queue
void print_queue(Queue* q) {
    Node* current = q->first;
    printf("Queue: ");
    while (current != NULL) {
        printf("%d > ", current->value);
        current = current->next;
    }
    printf("NULL\n");
}

// Example usage
int main() {
    Queue* q = (Queue*) malloc(sizeof(Queue));
    q->first = NULL;
    q->last = NULL;

    q = insert_node(q, 10);
    q = insert_node(q, 20);
    q = insert_node(q, 30);

    print_queue(q);

    q = remove_node(q);
    print_queue(q);

    free_queue(q);
    return 0;
}
```

Filas

○ IMPLEMENTAÇÃO DE FILA

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
39
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

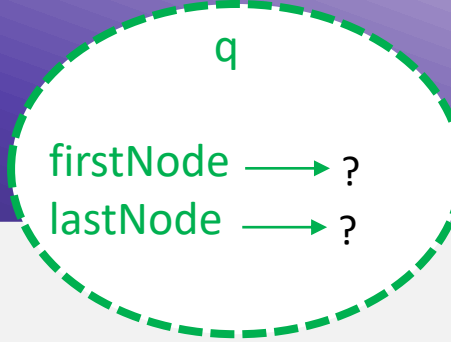
Filas

○ IMPLEMENTAÇÃO DE FILA

```
56 // Check if queue is empty
57 int is_empty(Queue* q) {
58     return (q->first == NULL);
59 }
60
61 // Free the queue
62 void free_queue(Queue* q) {
63     Node* current = q->first;
64     while (current != NULL) {
65         Node* temp = current;
66         current = current->next;
67         free(temp);
68     }
69     free(q);
70 }
71
72 // Print the queue
73 void print_queue(Queue* q) {
74     Node* current = q->first;
75     printf("Queue: ");
76     while (current != NULL) {
77         printf("%d -> ", current->value);
78         current = current->next;
79     }
80     printf("NULL\n");
81 }
```

Exemplo de insert

Filas

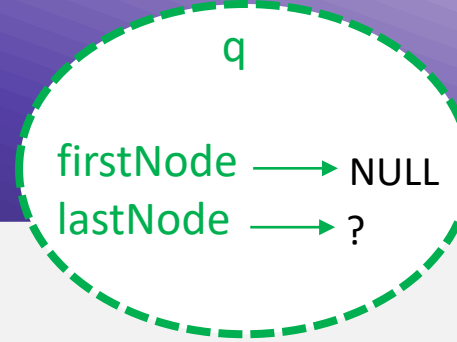


○ INSERT

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

```
10 // Queue structure  
11 typedef struct Queue {  
12     Node* first;  
13     Node* last;  
14 } Queue;
```

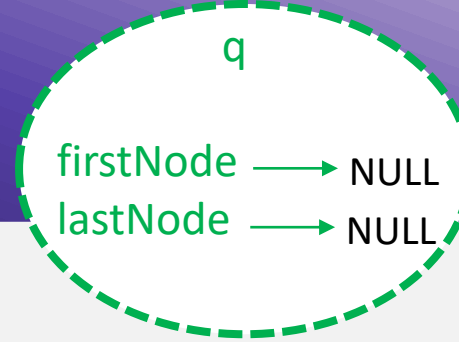
Filas



○ INSERT

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

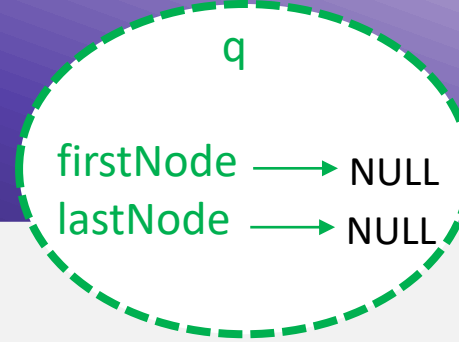
Filas



○ INSERT

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

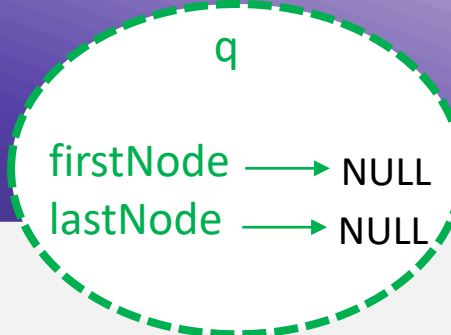
Filas



○ INSERT

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

Filas



○ INSERT

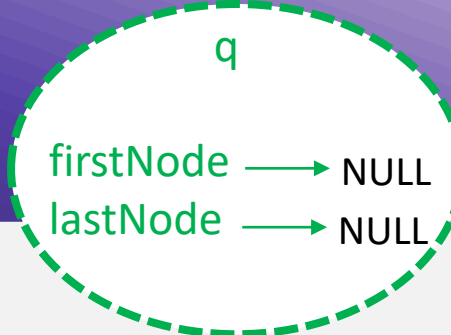
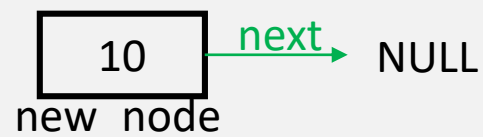
```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

```
16 // Create a new node  
17 Node* create_node(int value) {  
18     Node* new_node = (Node*) malloc(sizeof(Node));  
19     new_node->value = value;  
20     new_node->next = NULL;  
21     return new_node;  
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila  
25 Queue* insert_node(Queue* q, int value) {  
26     Node* new_node = create_node(value);  
27  
28     if (q->last == NULL) {  
29         // fila estava vazia  
30         q->first = new_node;  
31         q->last = new_node;  
32     } else {  
33         q->last->next = new_node;  
34         q->last = new_node;  
35     }  
36  
37     return q;  
38 }
```

Filas

○ INSERT



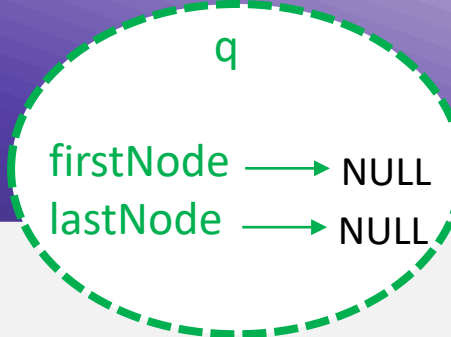
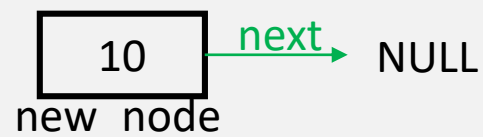
```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

```
16 // Create a new node  
17 Node* create_node(int value) {  
18     Node* new_node = (Node*) malloc(sizeof(Node));  
19     new_node->value = value;  
20     new_node->next = NULL;  
21     return new_node;  
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila  
25 Queue* insert_node(Queue* q, int value) {  
26     Node* new_node = create_node(value);  
27  
28     if (q->last == NULL) {  
29         // fila estava vazia  
30         q->first = new_node;  
31         q->last = new_node;  
32     } else {  
33         q->last->next = new_node;  
34         q->last = new_node;  
35     }  
36  
37     return q;  
38 }
```

Filas

○ INSERT



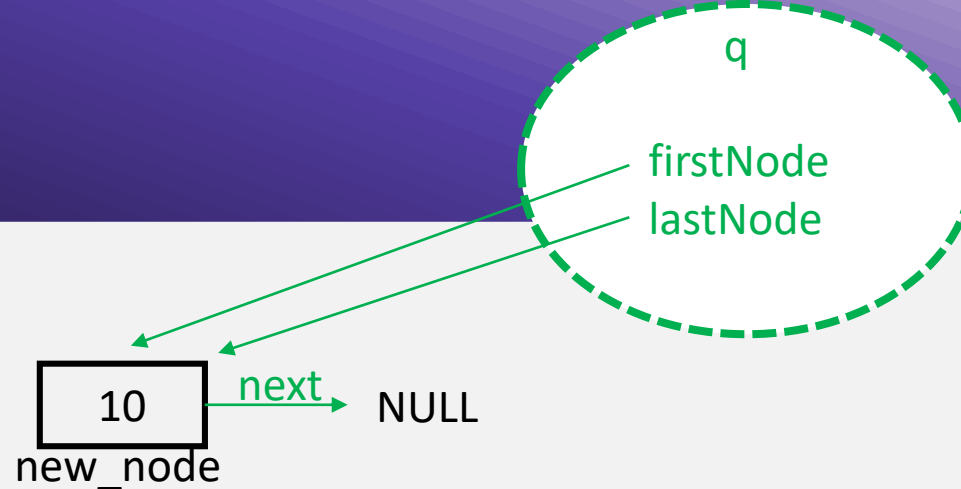
```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

```
16 // Create a new node  
17 Node* create_node(int value) {  
18     Node* new_node = (Node*) malloc(sizeof(Node));  
19     new_node->value = value;  
20     new_node->next = NULL;  
21     return new_node;  
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila  
25 Queue* insert_node(Queue* q, int value) {  
26     Node* new_node = create_node(value);  
27  
28     if (q->last == NULL) {  
29         // fila estava vazia  
30         q->first = new_node;  
31         q->last = new_node;  
32     } else {  
33         q->last->next = new_node;  
34         q->last = new_node;  
35     }  
36  
37     return q;  
38 }
```

Filas

○ INSERT



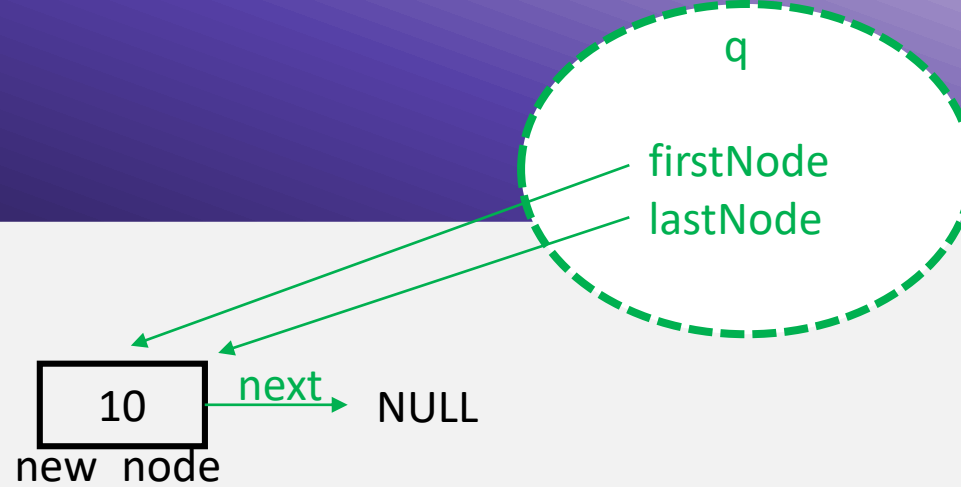
```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

```
16 // Create a new node  
17 Node* create_node(int value) {  
18     Node* new_node = (Node*) malloc(sizeof(Node));  
19     new_node->value = value;  
20     new_node->next = NULL;  
21     return new_node;  
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila  
25 Queue* insert_node(Queue* q, int value) {  
26     Node* new_node = create_node(value);  
27  
28     if (q->last == NULL) {  
29         // fila estava vazia  
30         q->first = new_node;  
31         q->last = new_node;  
32     } else {  
33         q->last->next = new_node;  
34         q->last = new_node;  
35     }  
36  
37     return q;  
38 }
```

Filas

○ INSERT



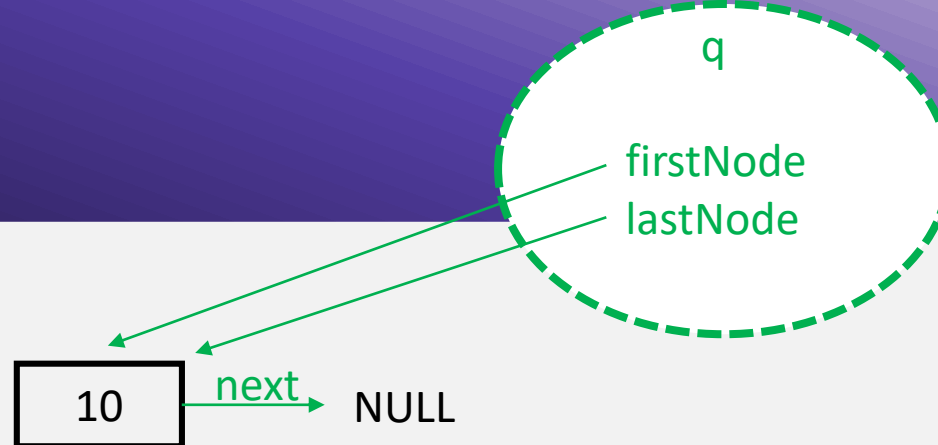
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87     q = insert_node(q, 10);
88     q = insert_node(q, 20);
89     q = insert_node(q, 30);
90     q = insert_node(q, 40);
91     q = insert_node(q, 50);
92
93     print_queue(q);
94
95     free_queue(q);
96     return 0;
97 }
98
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



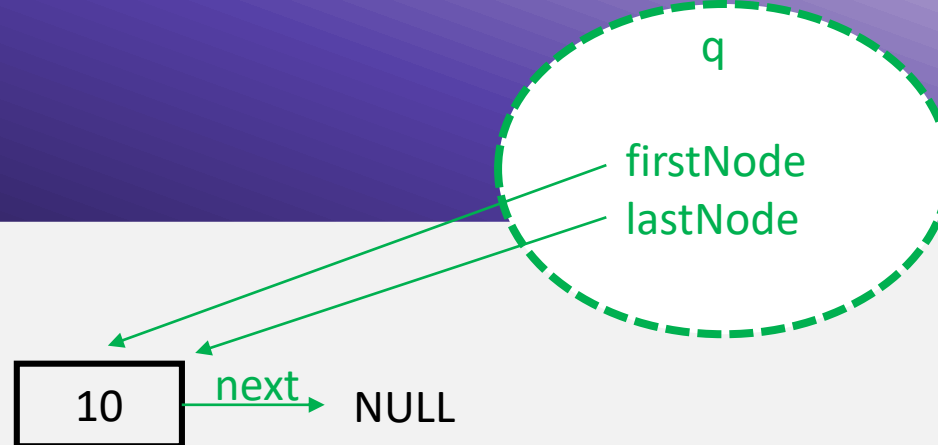
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87     q = insert_node(q, 10);
88     q = insert_node(q, 20);
89     q = insert_node(q, 30);
90     q = insert_node(q, 40);
91     q = insert_node(q, 50);
92
93     print_queue(q);
94
95     free_queue(q);
96     return 0;
97 }
98
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



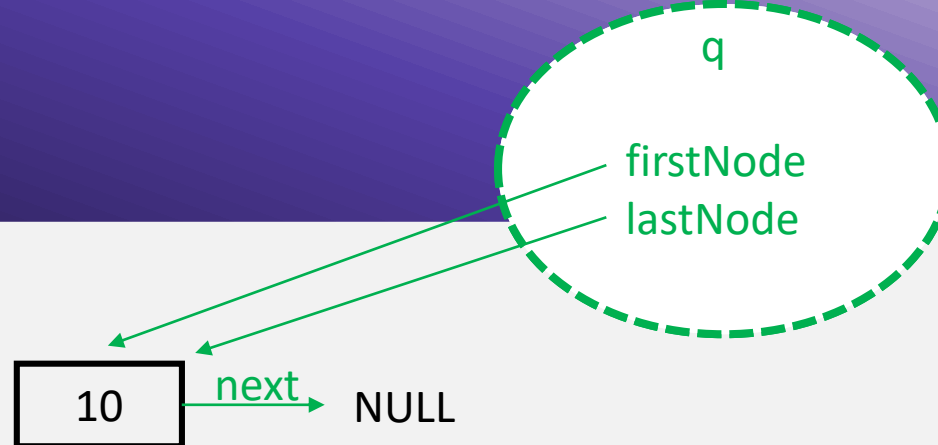
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



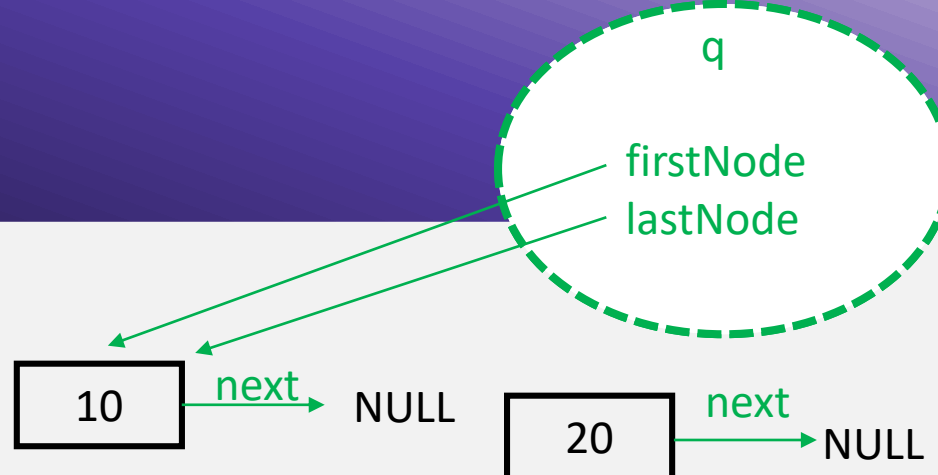
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

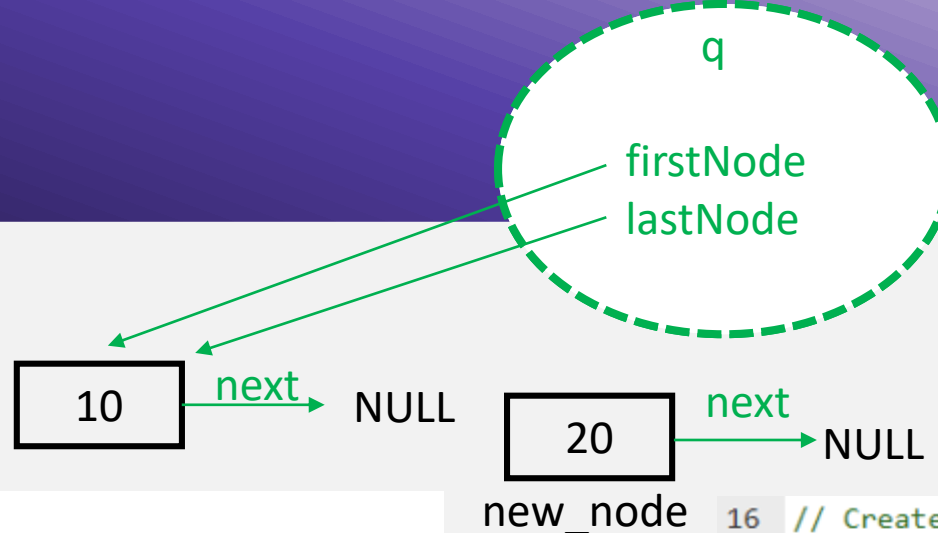
new_node

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



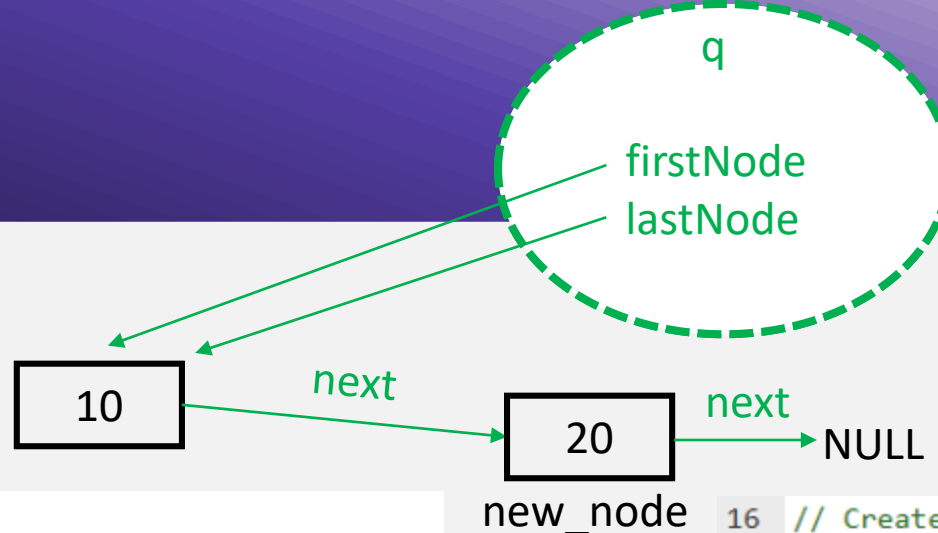
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



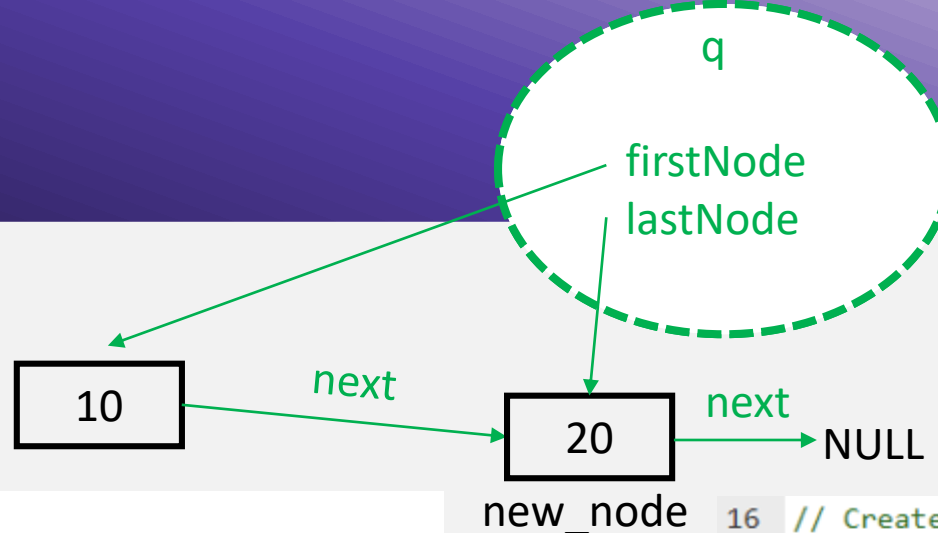
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



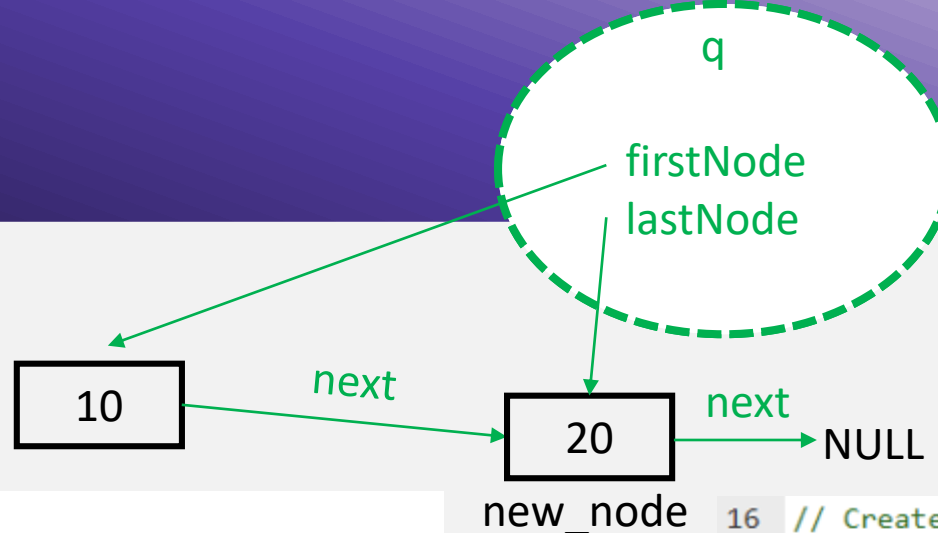
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



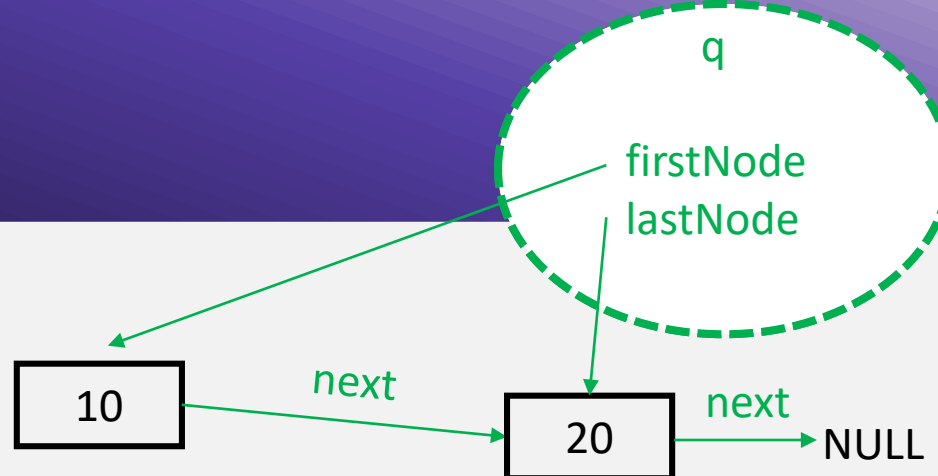
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



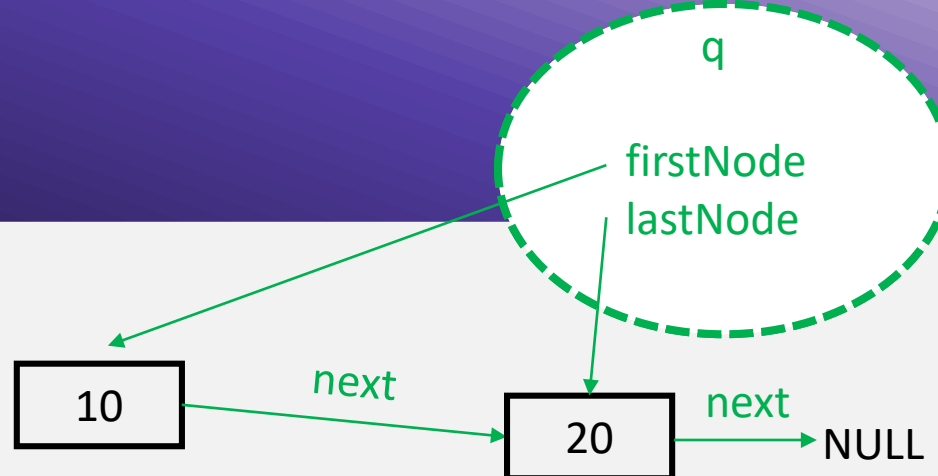
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



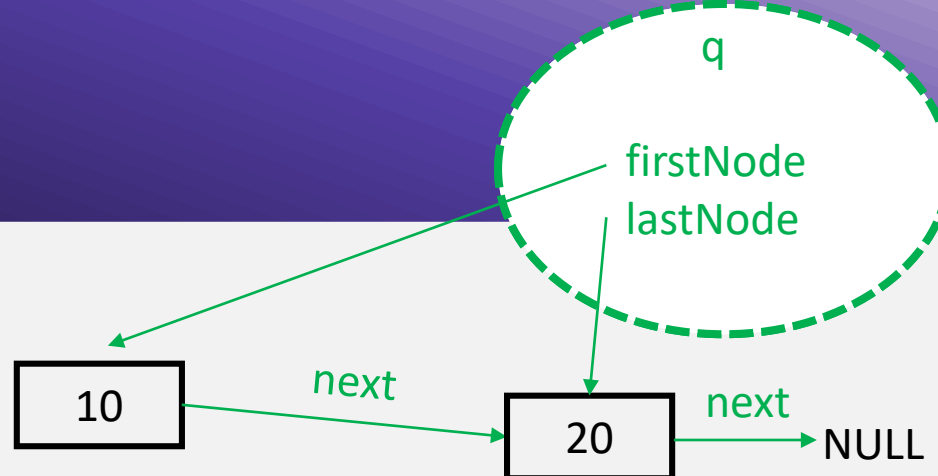
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



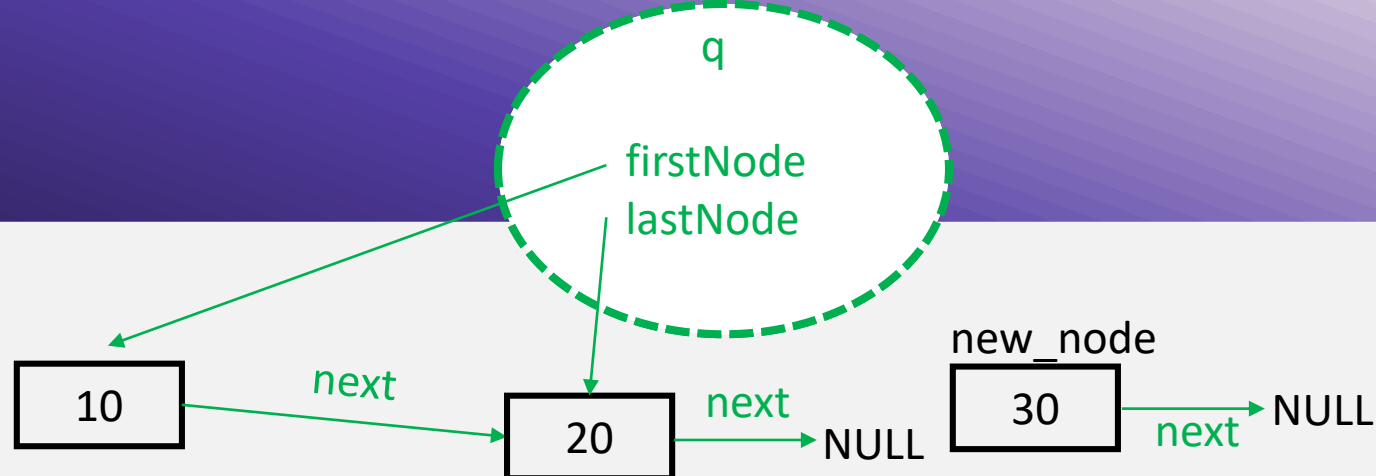
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



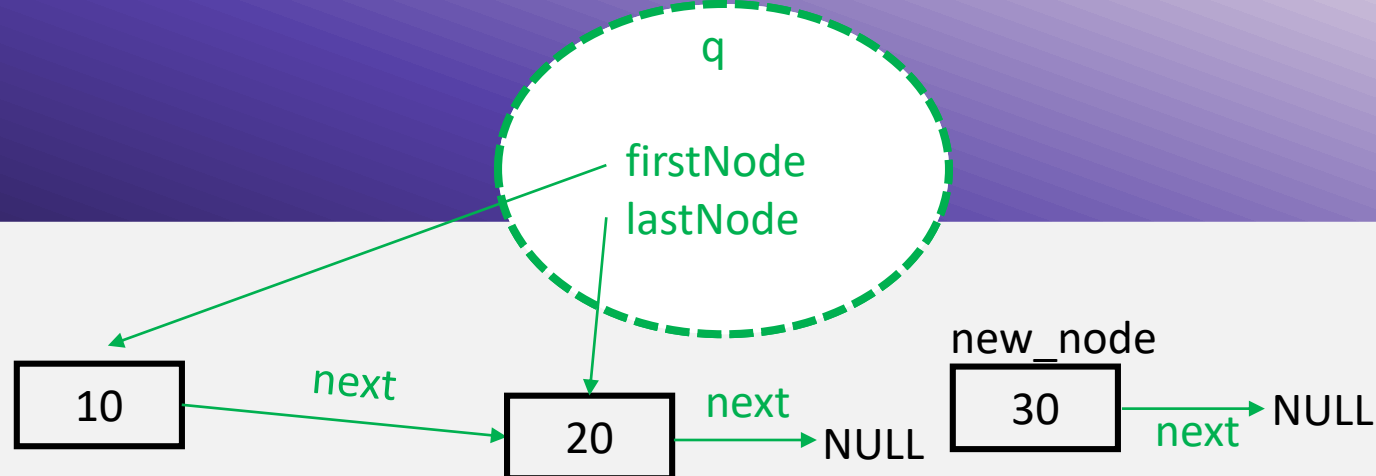
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



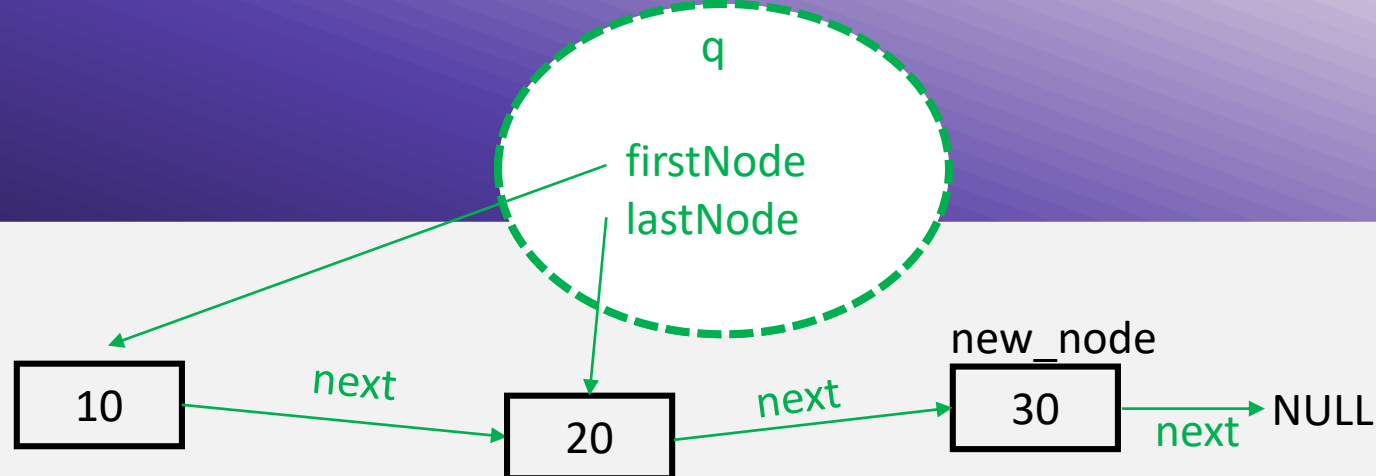
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



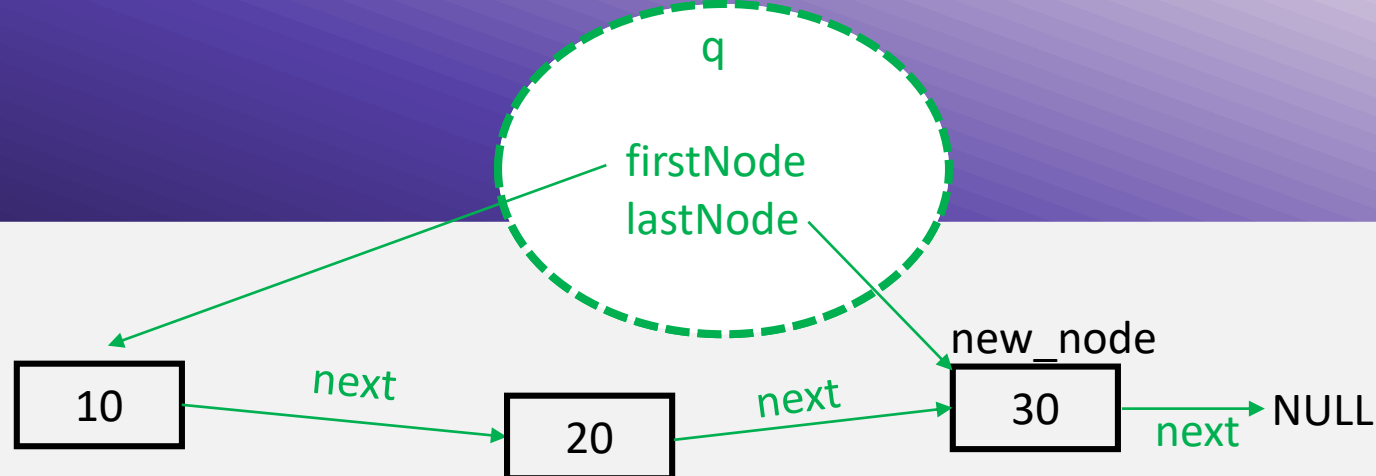
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



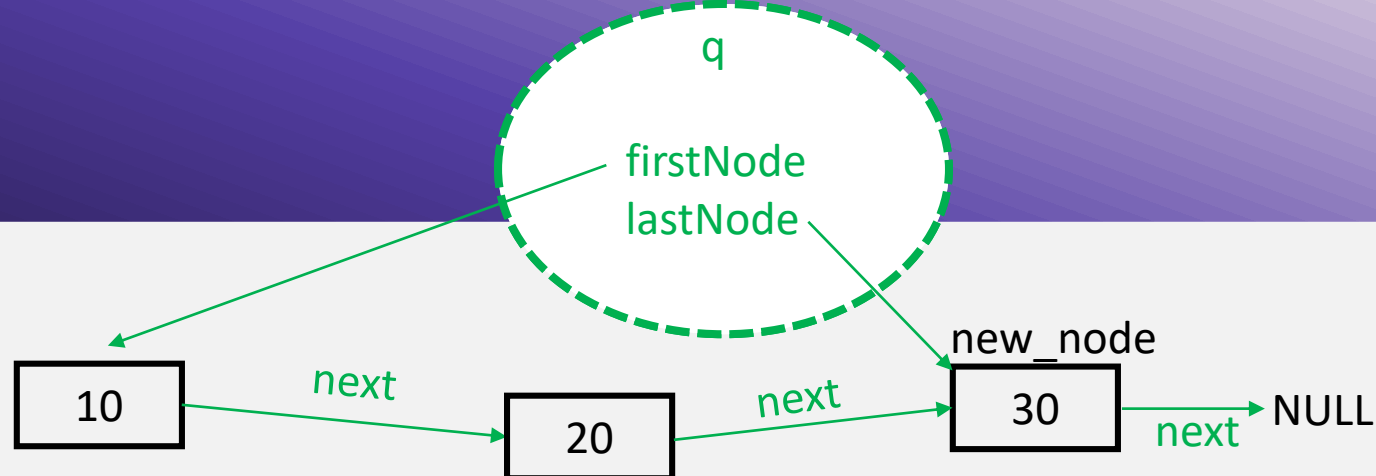
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35
36     }
37     return q;
38 }
```

Filas

○ INSERT



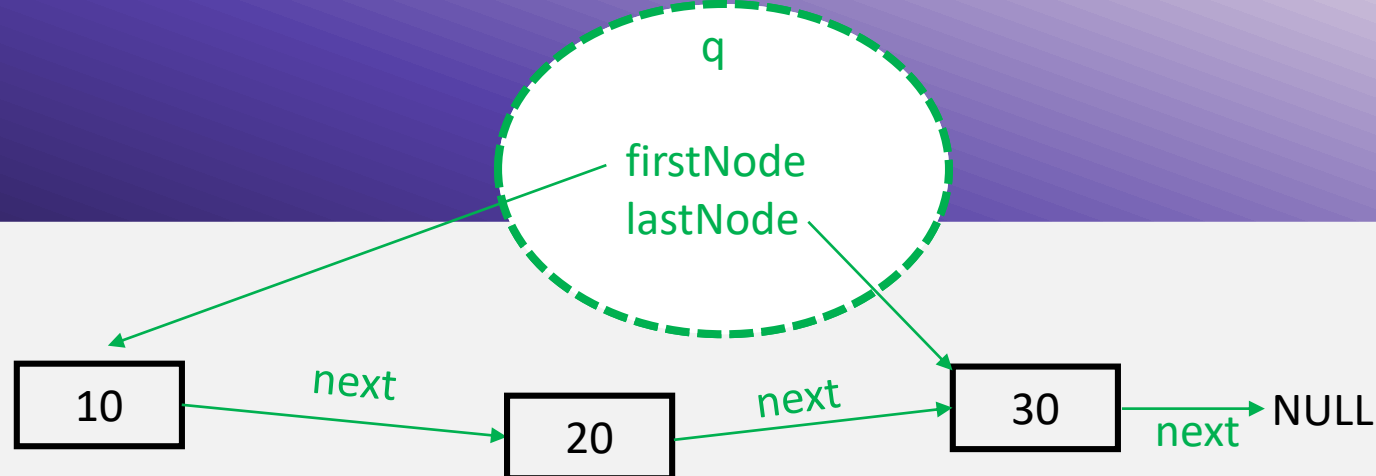
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



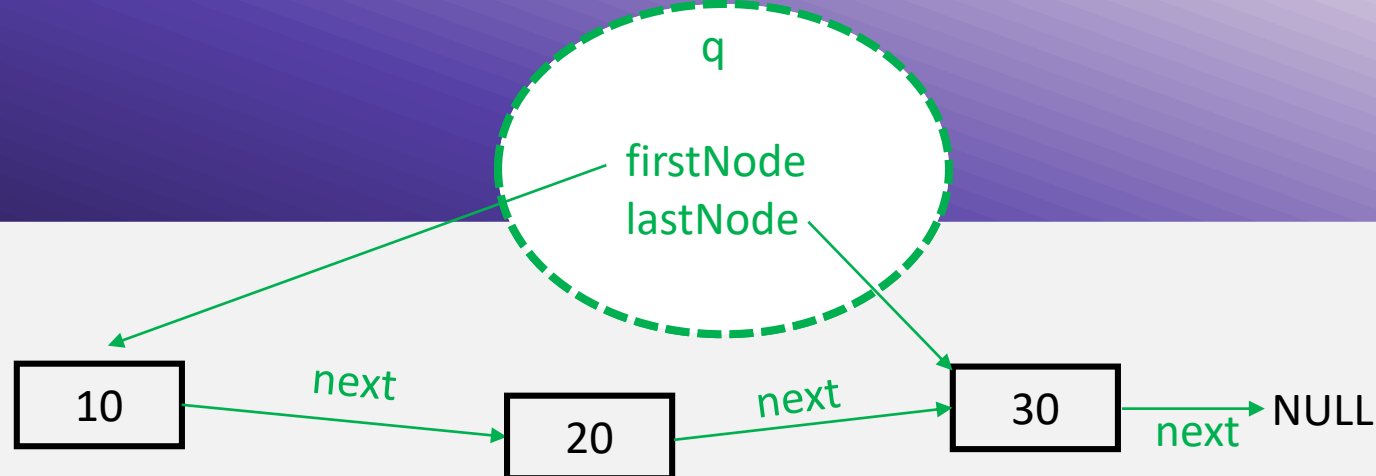
```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

```
16 // Create a new node  
17 Node* create_node(int value) {  
18     Node* new_node = (Node*) malloc(sizeof(Node));  
19     new_node->value = value;  
20     new_node->next = NULL;  
21     return new_node;  
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila  
25 Queue* insert_node(Queue* q, int value) {  
26     Node* new_node = create_node(value);  
27  
28     if (q->last == NULL) {  
29         // fila estava vazia  
30         q->first = new_node;  
31         q->last = new_node;  
32     } else {  
33         q->last->next = new_node;  
34         q->last = new_node;  
35     }  
36  
37     return q;  
38 }
```

Filas

○ INSERT



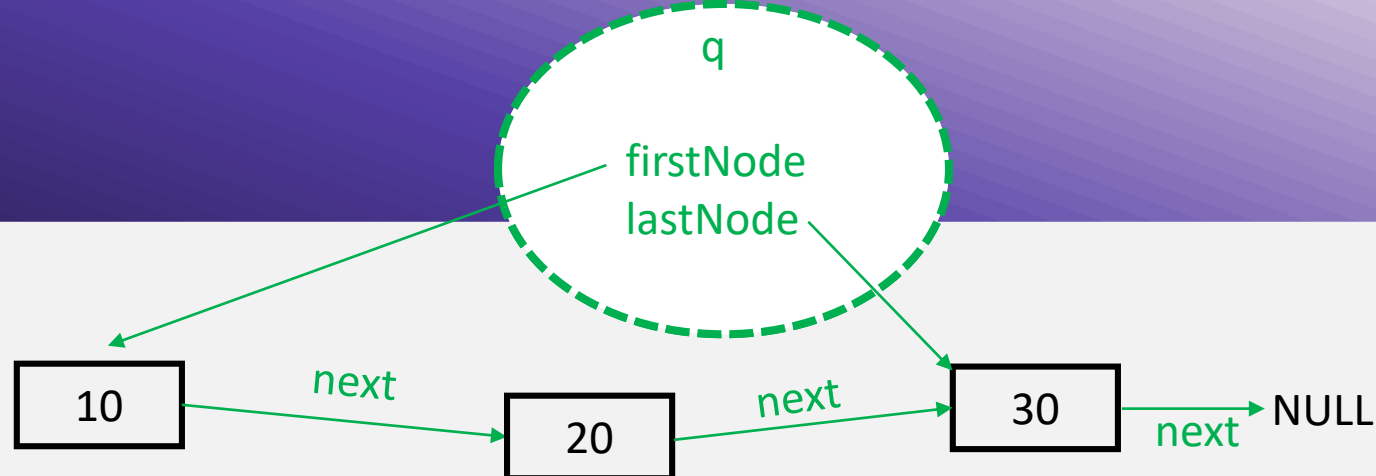
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



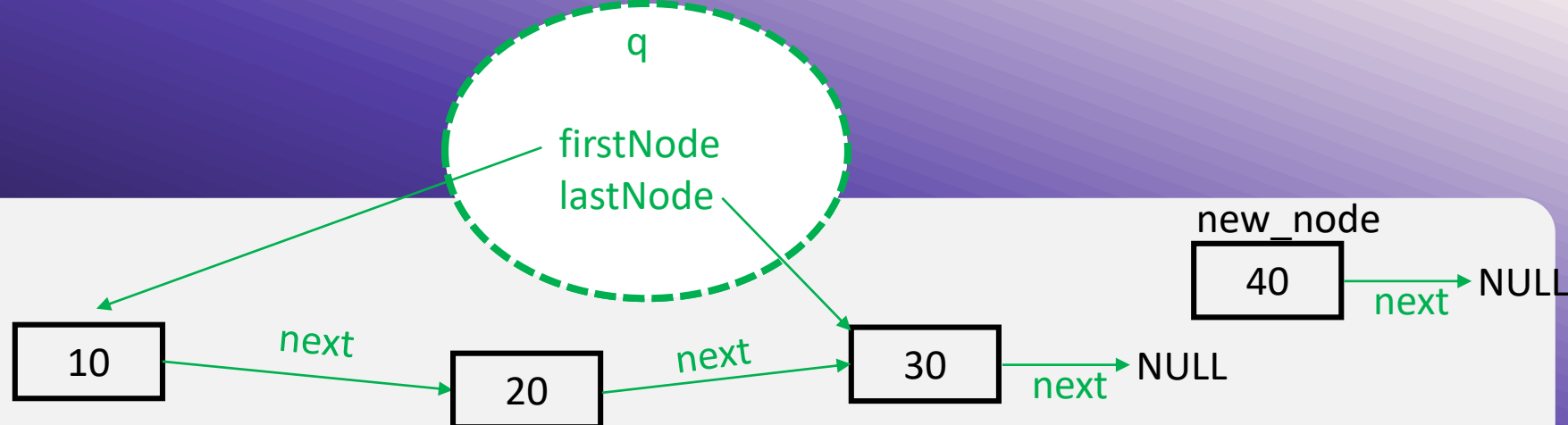
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



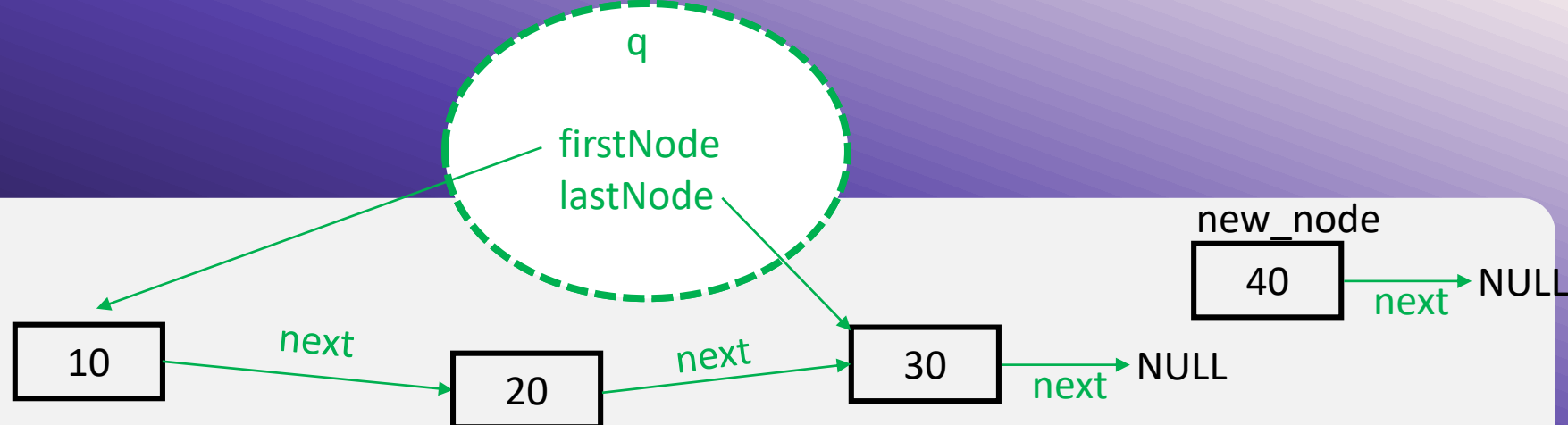
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



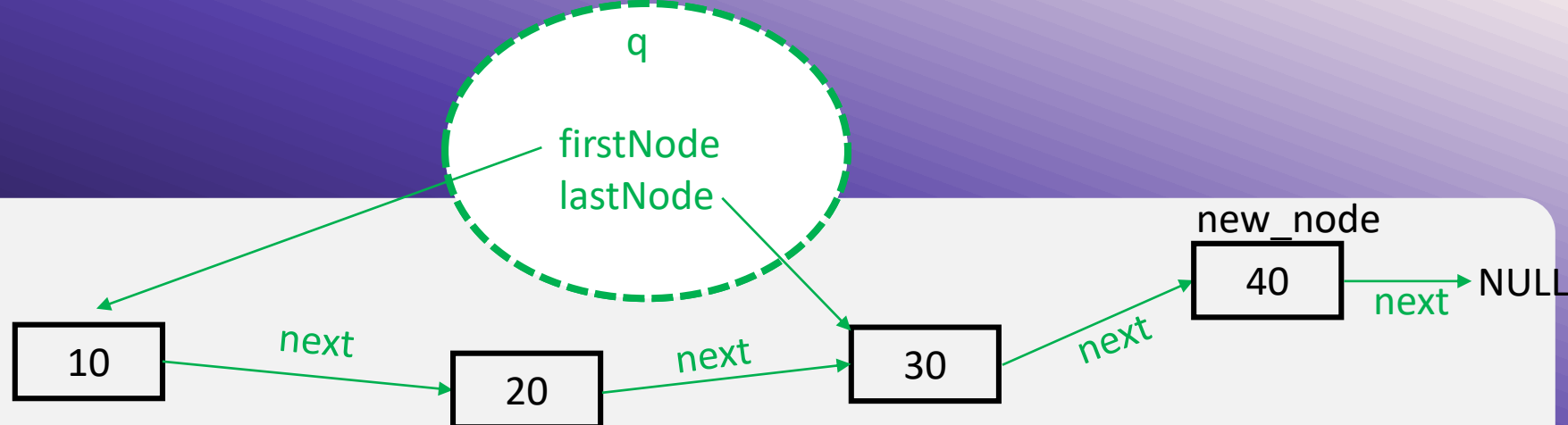
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



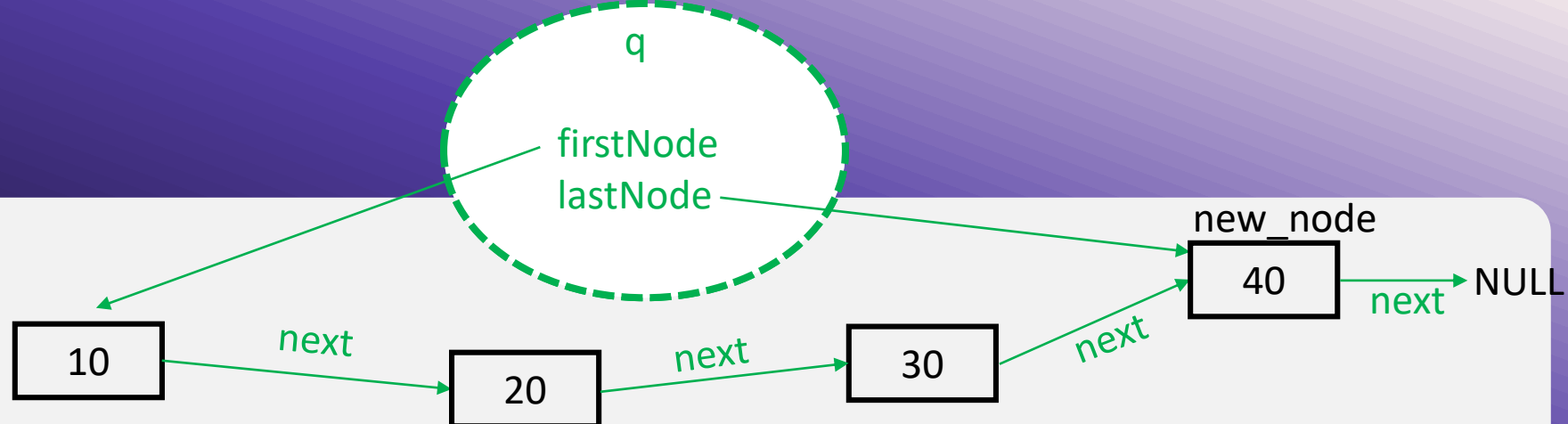
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



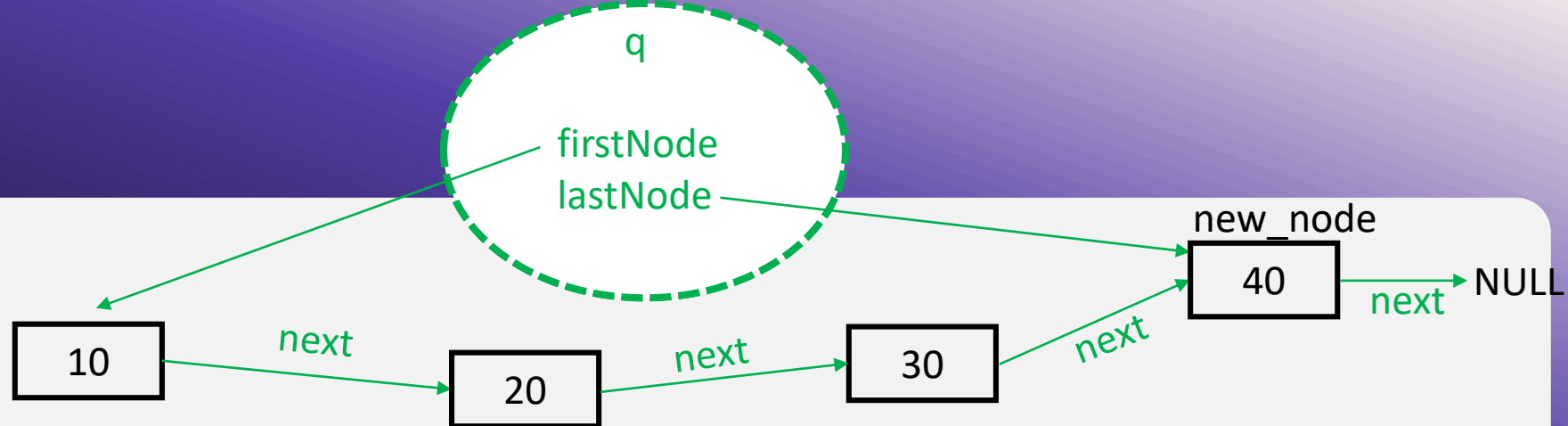
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



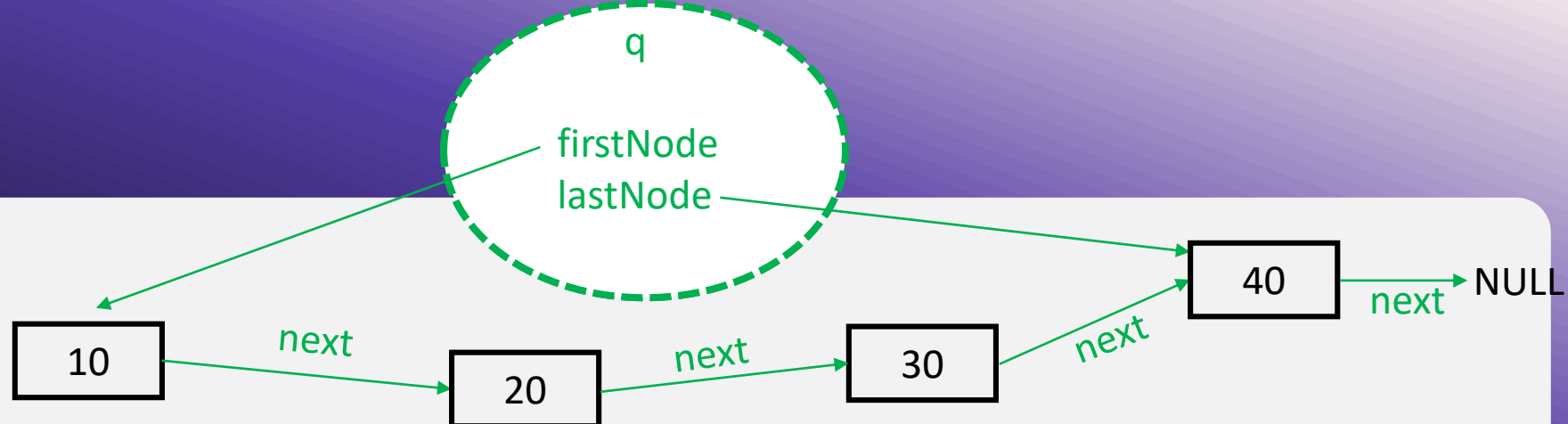
```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

```
16 // Create a new node  
17 Node* create_node(int value) {  
18     Node* new_node = (Node*) malloc(sizeof(Node));  
19     new_node->value = value;  
20     new_node->next = NULL;  
21     return new_node;  
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila  
25 Queue* insert_node(Queue* q, int value) {  
26     Node* new_node = create_node(value);  
27  
28     if (q->last == NULL) {  
29         // fila estava vazia  
30         q->first = new_node;  
31         q->last = new_node;  
32     } else {  
33         q->last->next = new_node;  
34         q->last = new_node;  
35     }  
36  
37     return q;  
38 }
```

Filas

○ INSERT



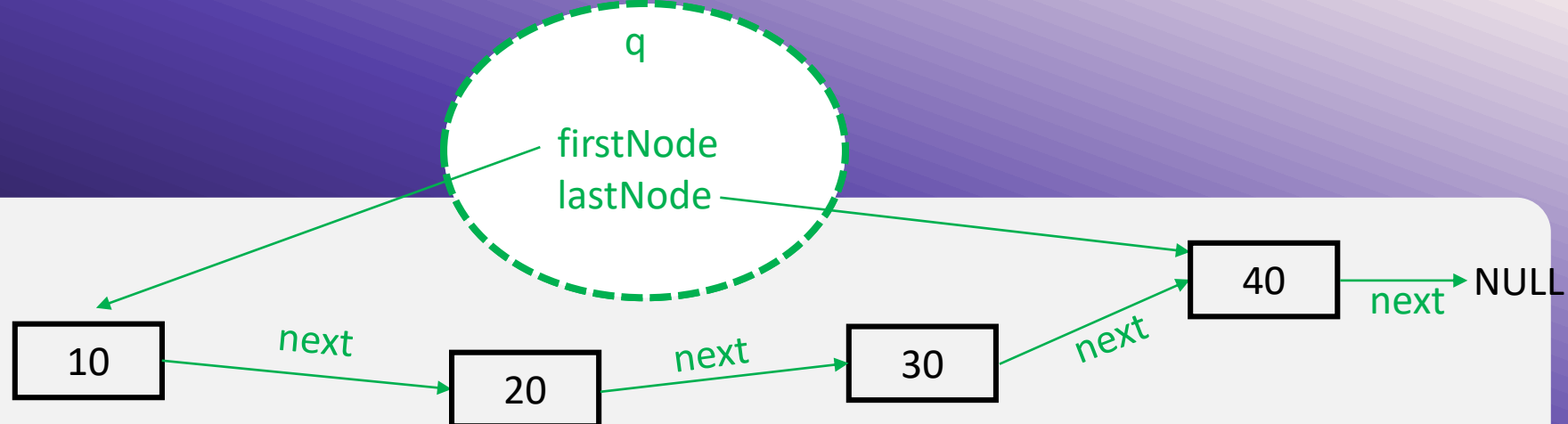
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



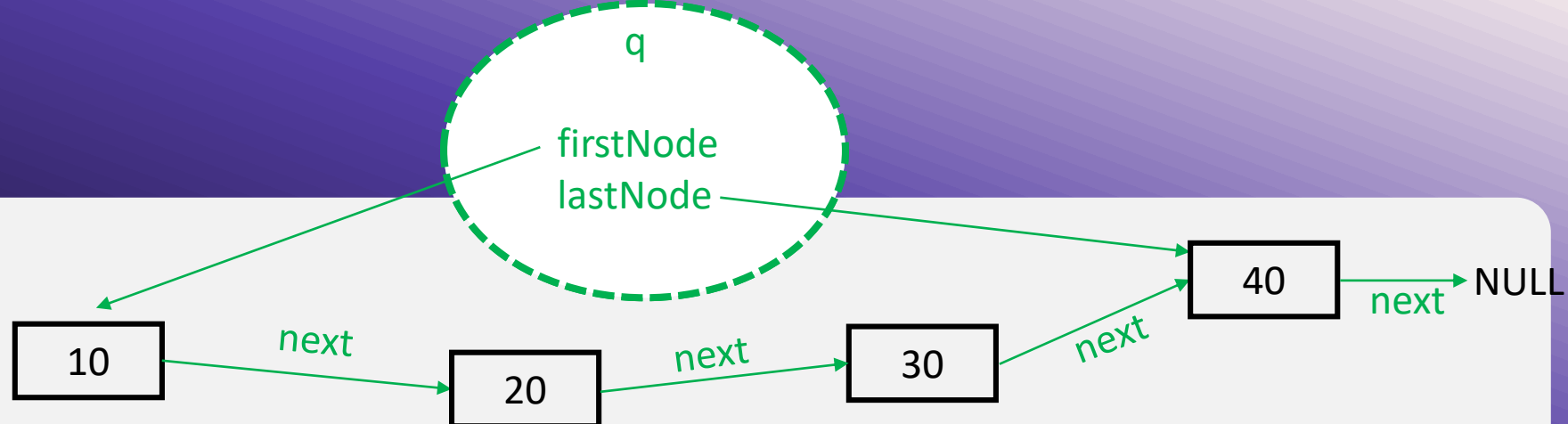
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



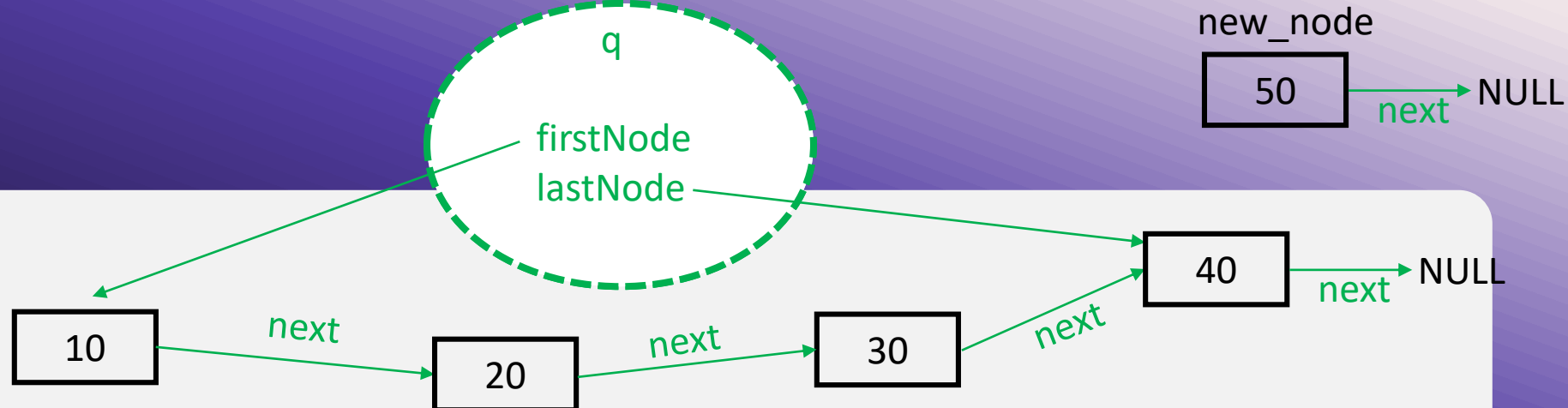
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



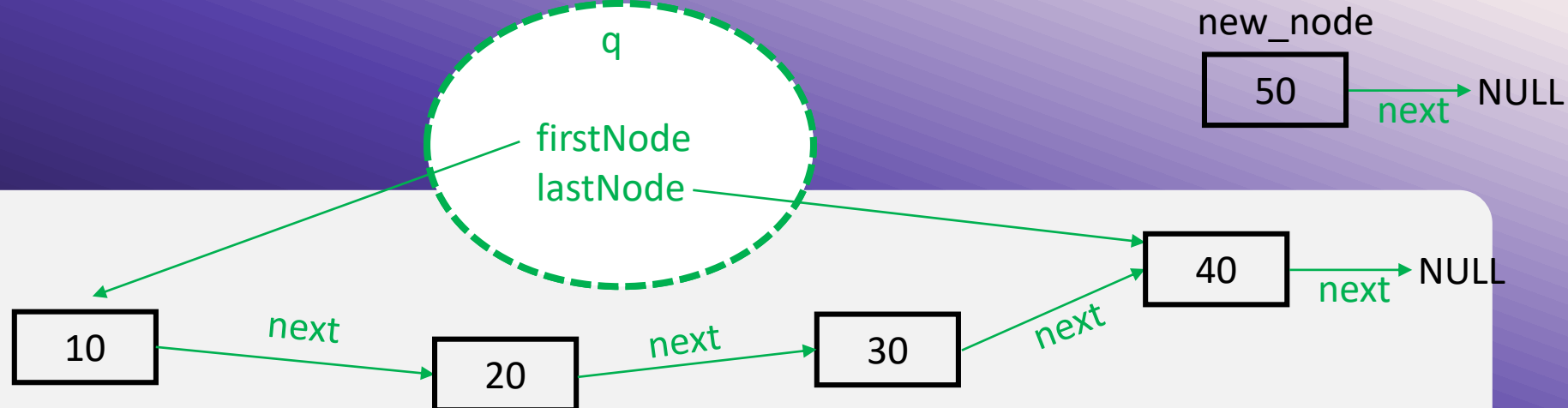
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



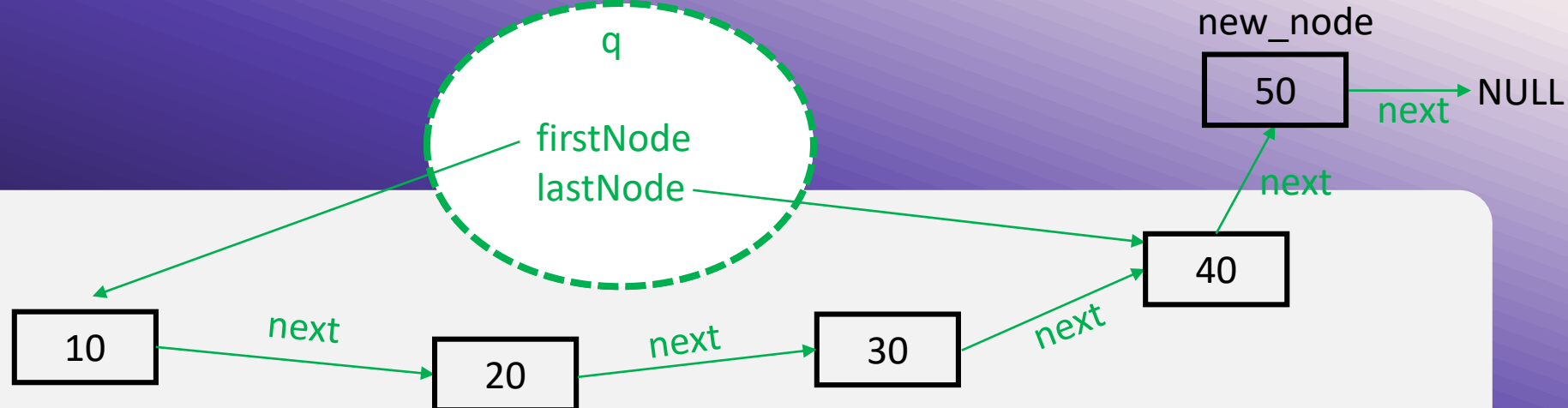
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



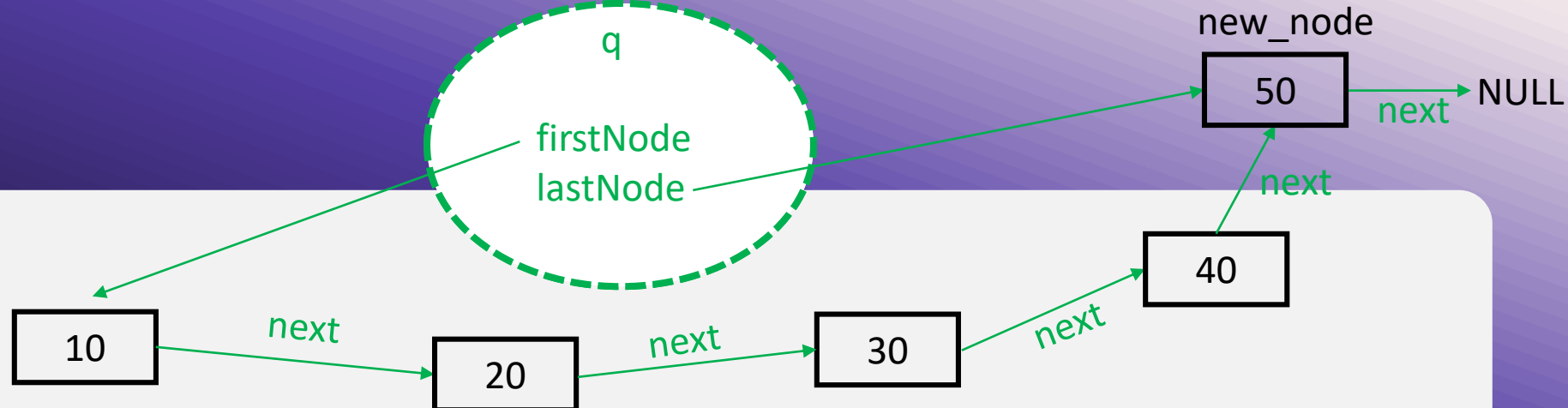
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



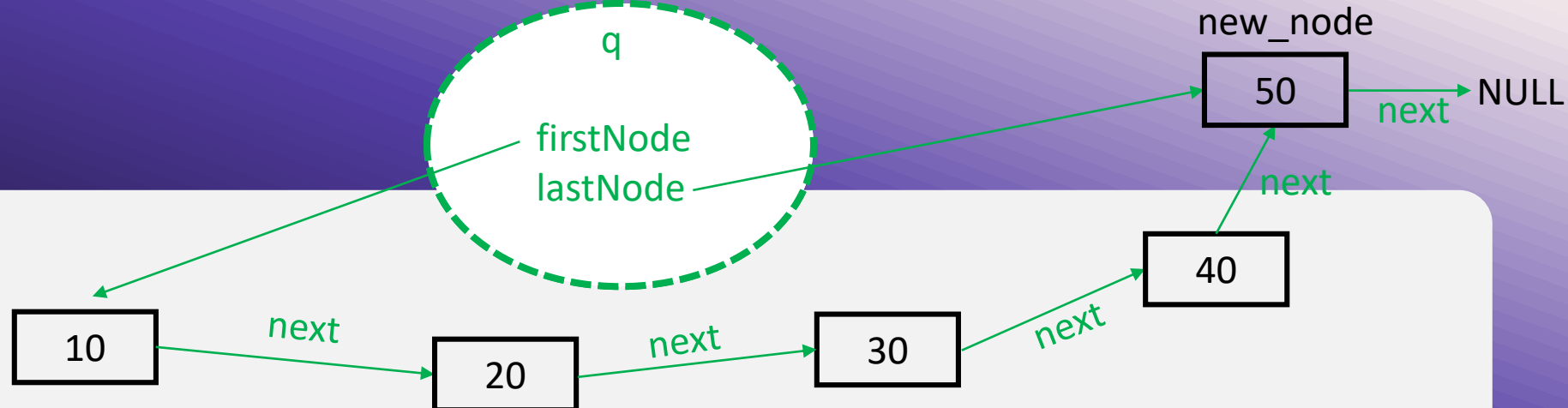
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



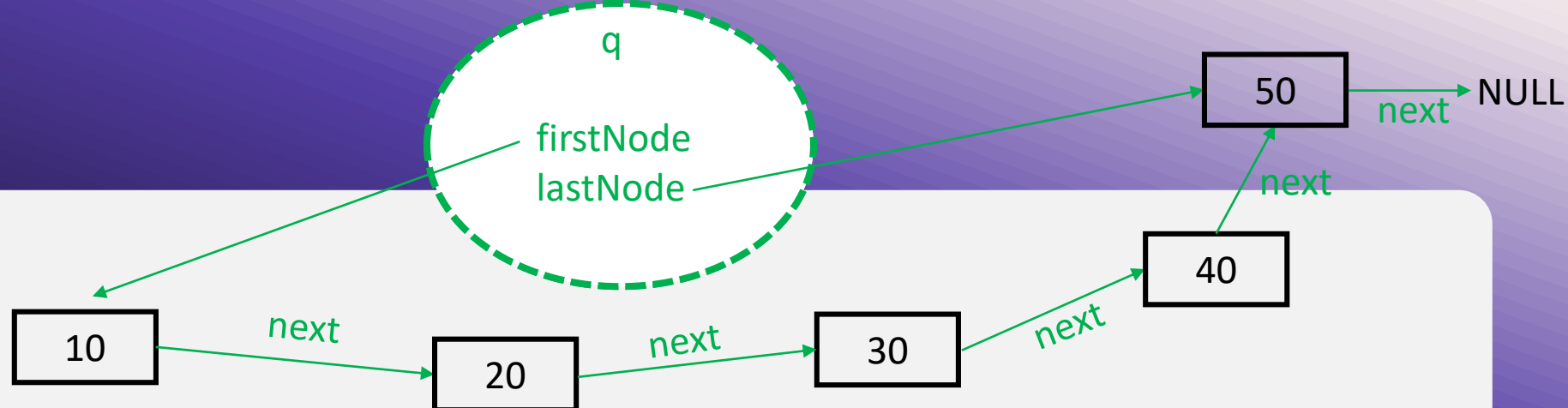
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



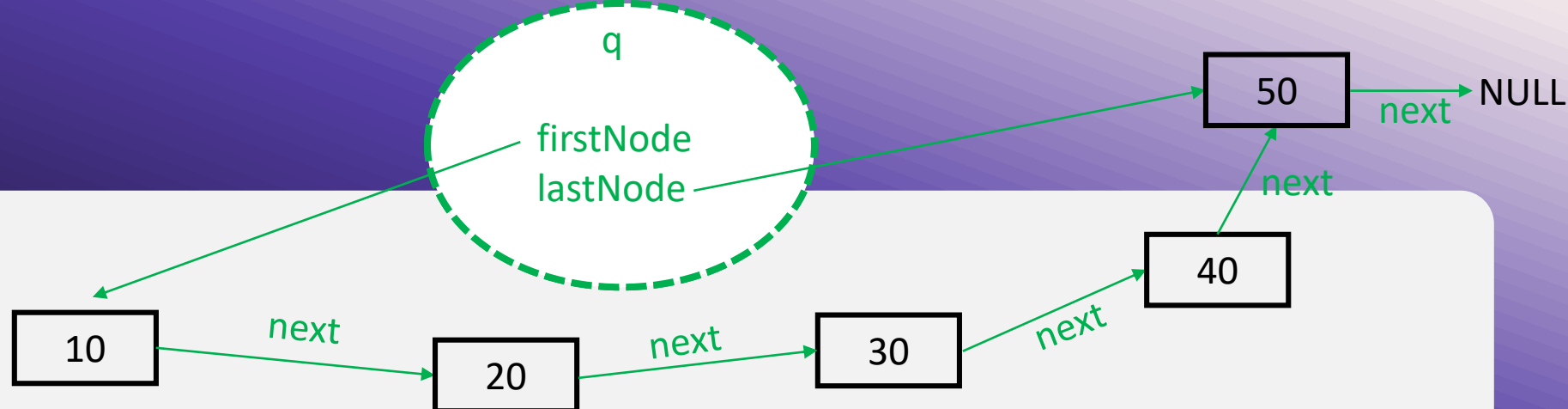
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

Filas

○ INSERT



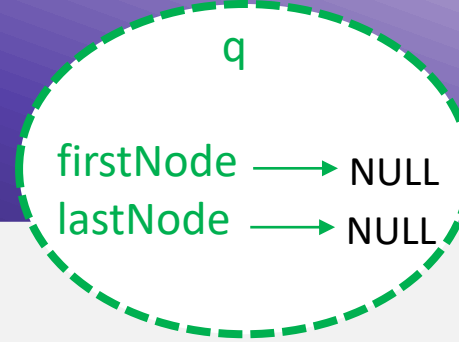
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     print_queue(q);
95
96     free_queue(q);
97     return 0;
98 }
```

```
16 // Create a new node
17 Node* create_node(int value) {
18     Node* new_node = (Node*) malloc(sizeof(Node));
19     new_node->value = value;
20     new_node->next = NULL;
21     return new_node;
22 }
```

```
24 // Insert (enqueue) - insere no fim da fila
25 Queue* insert_node(Queue* q, int value) {
26     Node* new_node = create_node(value);
27
28     if (q->last == NULL) {
29         // fila estava vazia
30         q->first = new_node;
31         q->last = new_node;
32     } else {
33         q->last->next = new_node;
34         q->last = new_node;
35     }
36
37     return q;
38 }
```

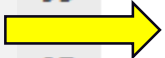
```
Queue: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
```

Filas



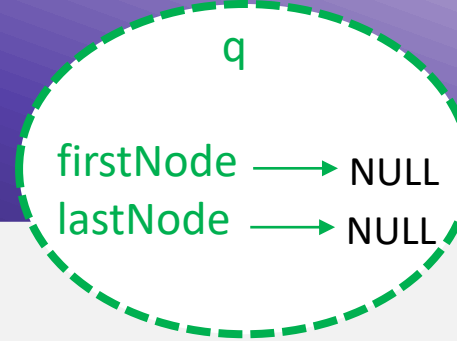
○ INSERT

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```



```
Queue: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
```

Filas



○ INSERT

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     print_queue(q);  
95  
96     free_queue(q);  
97     return 0;  
98 }
```

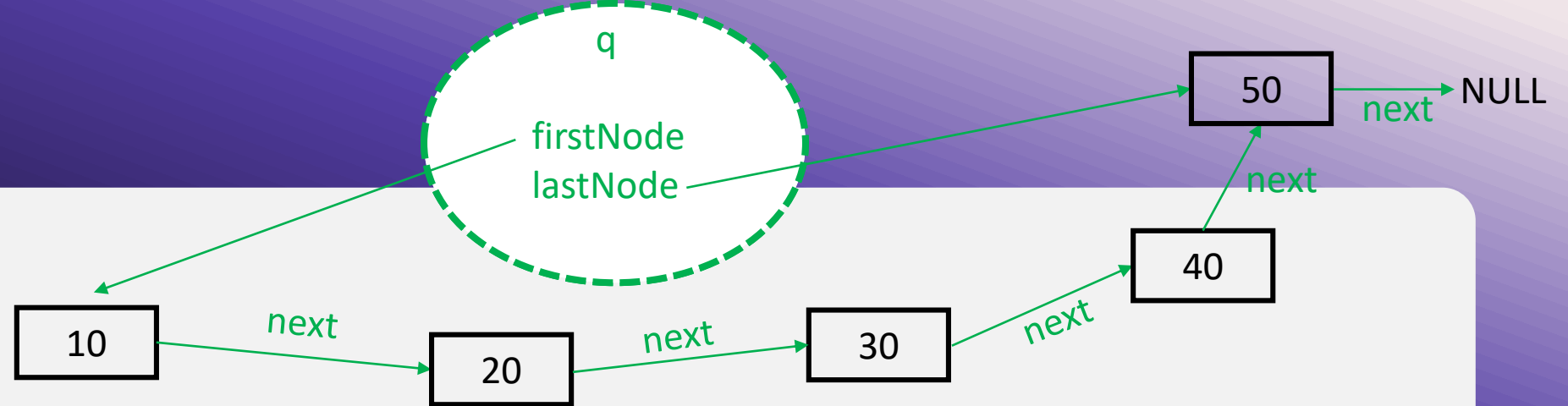
A screenshot of a terminal window with a black background and green text. The first line shows the output of the print_queue function: "Queue: 10 -> 20 -> 30 -> 40 -> 50 -> NULL". The second line shows the program's exit message: "...Program finished with exit code 0". The third line shows a prompt for the user to press ENTER to exit the console.

```
Queue: 10 -> 20 -> 30 -> 40 -> 50 -> NULL  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Exemplo de remove

Filas

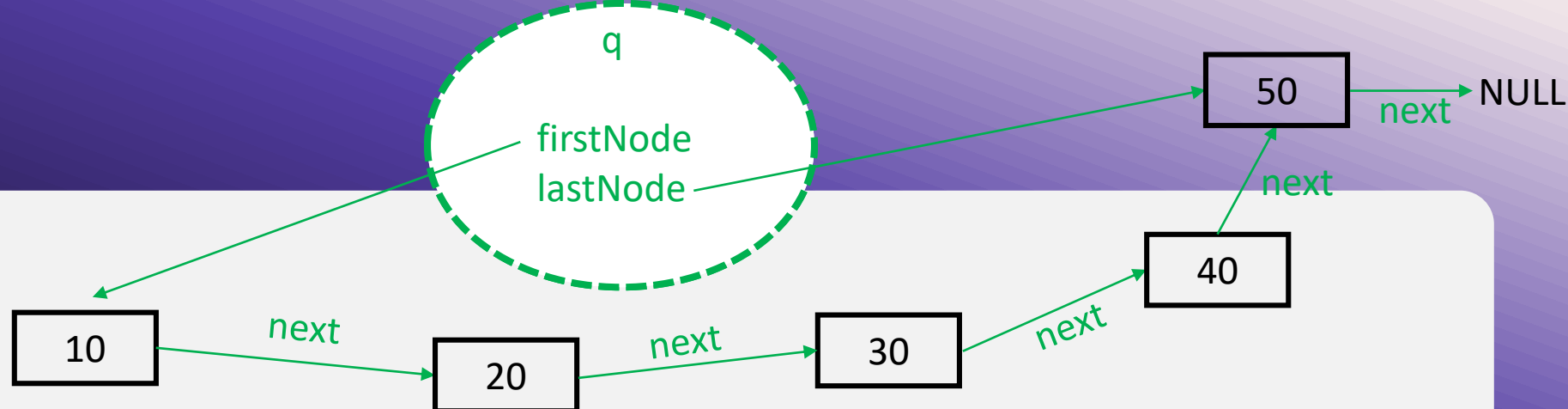
○ REMOVE



```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     q = remove_node(q);  
95     q = remove_node(q);  
96     q = remove_node(q);  
97  
98     print_queue(q);  
99  
100    free_queue(q);  
101    return 0;  
102 }
```

Filas

○ REMOVE

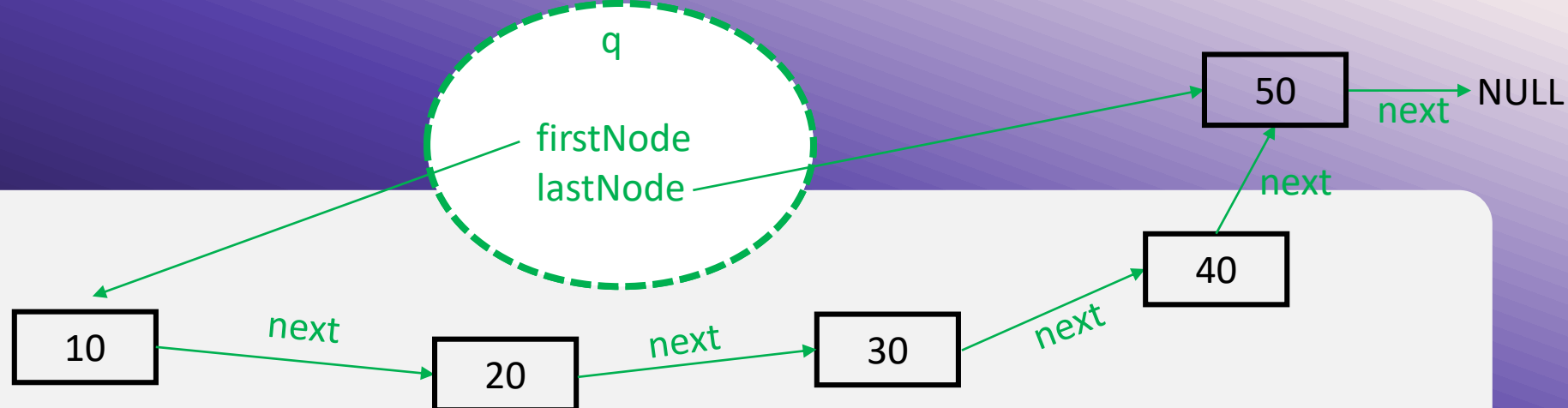


```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

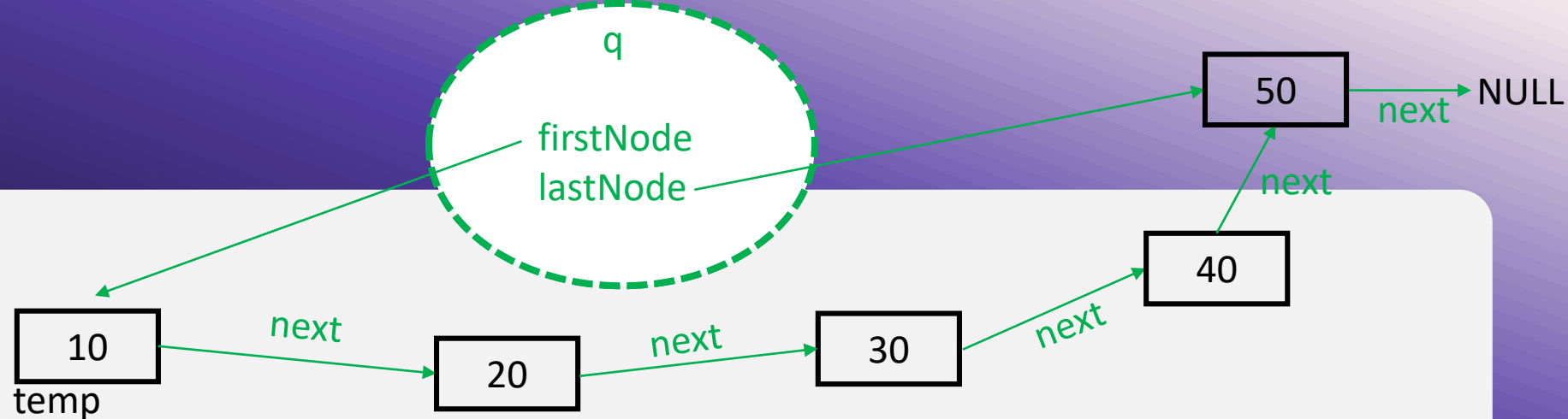


```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

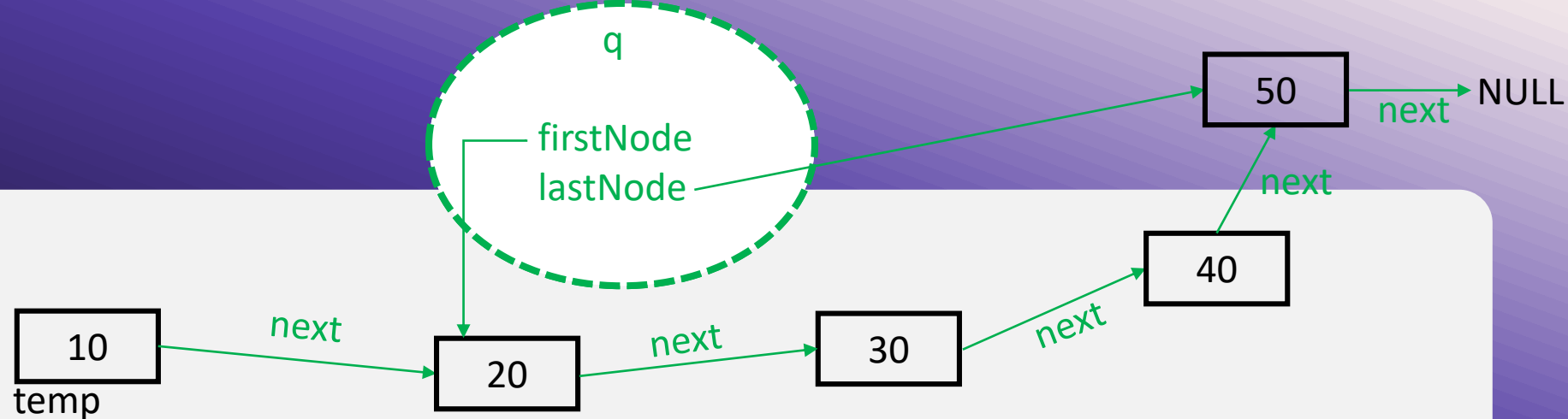


```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE



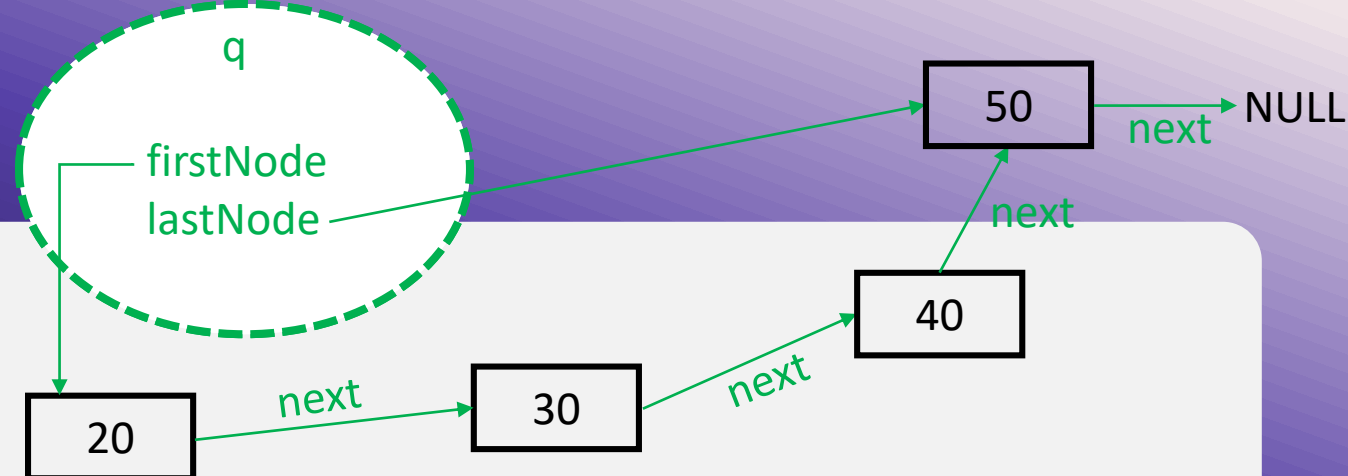
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     q = remove_node(q);  
95     q = remove_node(q);  
96     q = remove_node(q);  
97  
98     print_queue(q);  
99  
100    free_queue(q);  
101    return 0;  
102 }
```

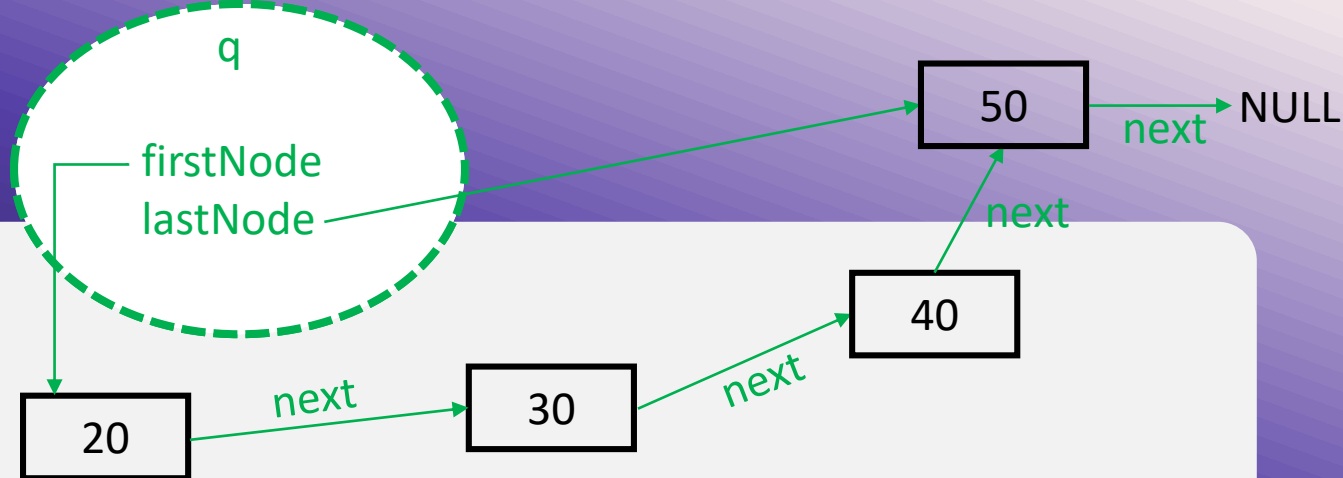


```
40 // Remove (dequeue) - remove do início da fila  
41 Queue* remove_node(Queue* q) {  
42     if (q->first == NULL) return q; // fila vazia  
43  
44     Node* temp = q->first;  
45     q->first = q->first->next;  
46  
47     if (q->first == NULL) {  
48         // fila ficou vazia  
49         q->last = NULL;  
50     }  
51  
52     free(temp);  
53     return q;  
54 }
```

Filas

○ REMOVE

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     q = remove_node(q);  
95     q = remove_node(q);  
96     q = remove_node(q);  
97  
98     print_queue(q);  
99  
100    free_queue(q);  
101    return 0;  
102 }
```

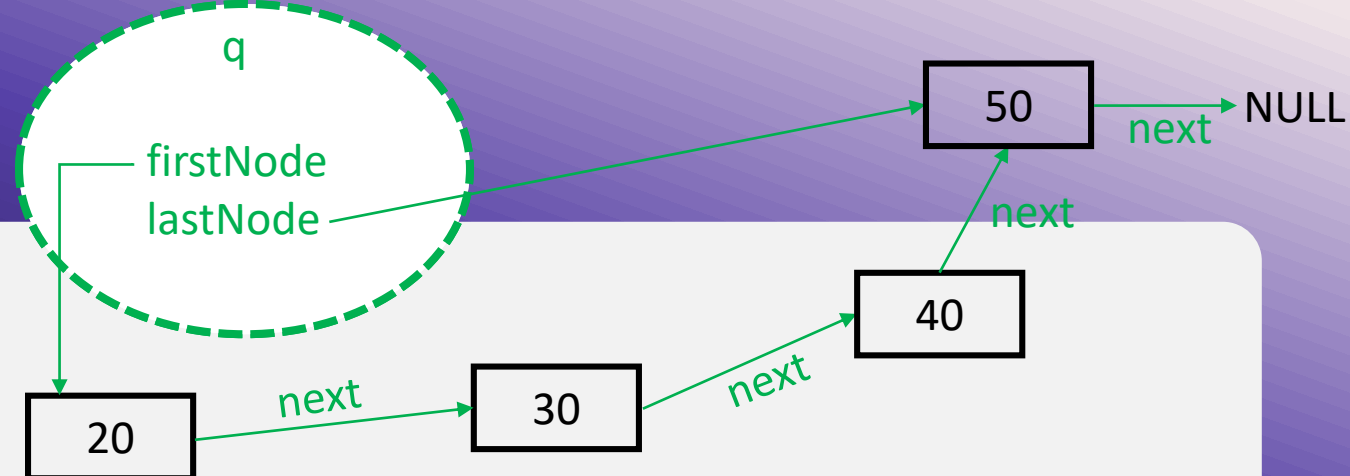


```
40 // Remove (dequeue) - remove do início da fila  
41 Queue* remove_node(Queue* q) {  
42     if (q->first == NULL) return q; // fila vazia  
43  
44     Node* temp = q->first;  
45     q->first = q->first->next;  
46  
47     if (q->first == NULL) {  
48         // fila ficou vazia  
49         q->last = NULL;  
50     }  
51  
52     free(temp);  
53     return q;  
54 }
```

Filas

○ REMOVE

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     q = remove_node(q);  
95     q = remove_node(q);  
96     q = remove_node(q);  
97  
98     print_queue(q);  
99  
100    free_queue(q);  
101    return 0;  
102 }
```

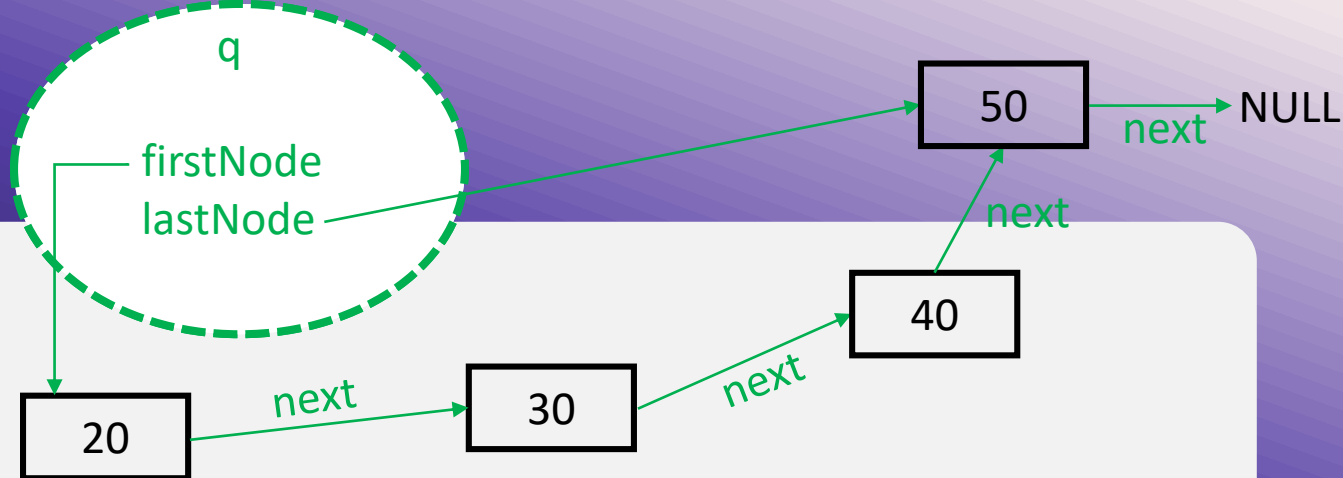


```
40 // Remove (dequeue) - remove do início da fila  
41 Queue* remove_node(Queue* q) {  
42     if (q->first == NULL) return q; // fila vazia  
43  
44     Node* temp = q->first;  
45     q->first = q->first->next;  
46  
47     if (q->first == NULL) {  
48         // fila ficou vazia  
49         q->last = NULL;  
50     }  
51  
52     free(temp);  
53     return q;  
54 }
```

Filas

○ REMOVE

```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

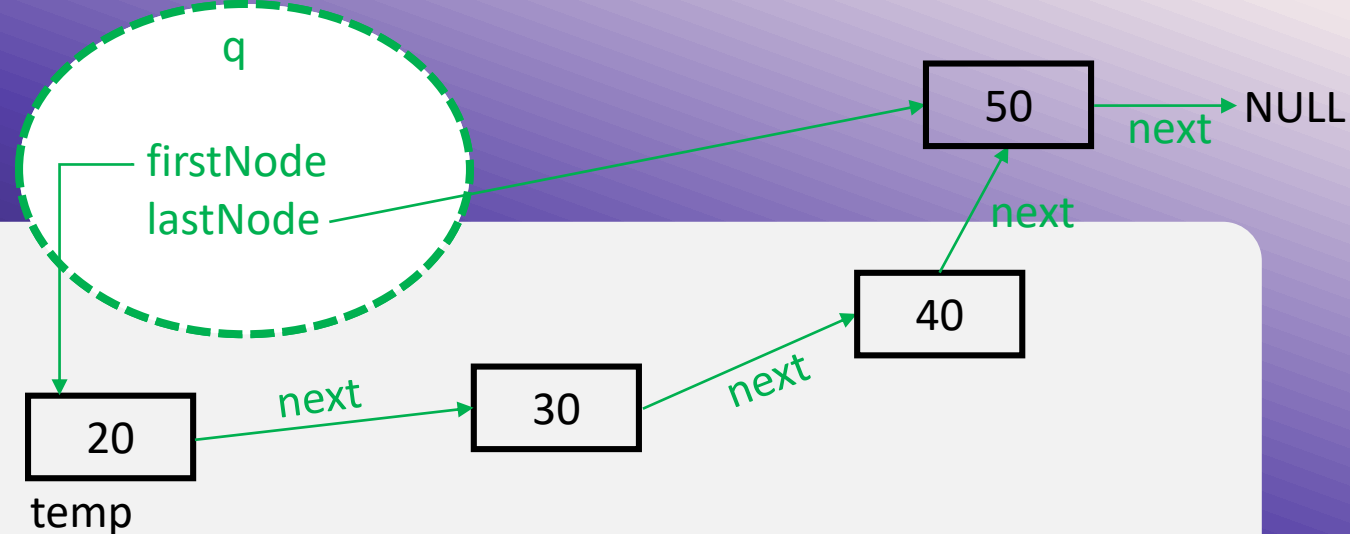


```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

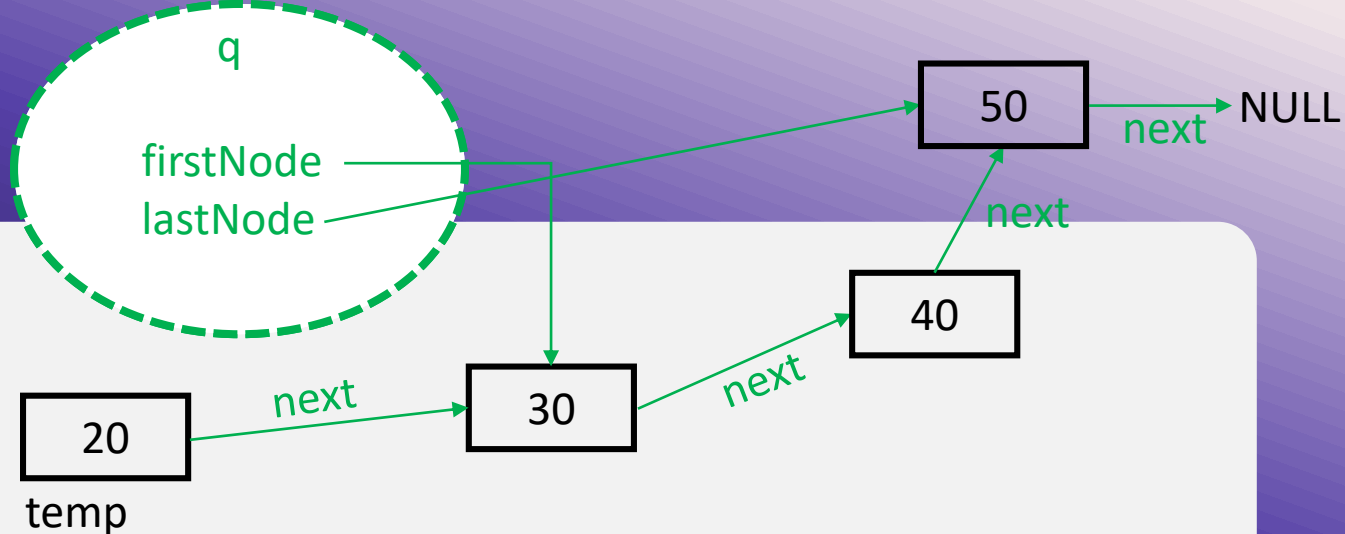


```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     q = remove_node(q);  
95     q = remove_node(q);  
96     q = remove_node(q);  
97  
98     print_queue(q);  
99  
100    free_queue(q);  
101    return 0;  
102 }
```

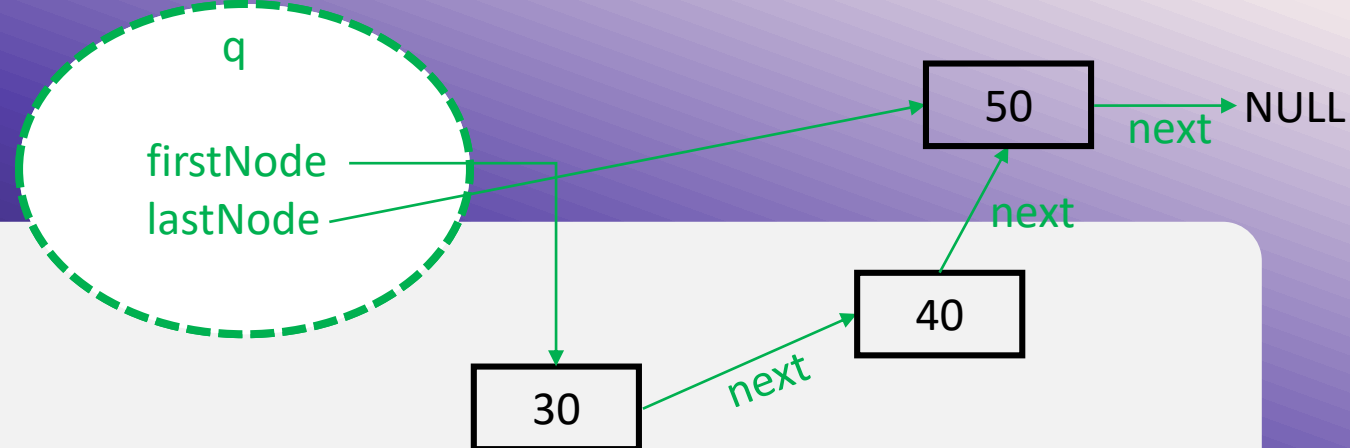


```
40 // Remove (dequeue) - remove do início da fila  
41 Queue* remove_node(Queue* q) {  
42     if (q->first == NULL) return q; // fila vazia  
43  
44     Node* temp = q->first;  
45     q->first = q->first->next;  
46  
47     if (q->first == NULL) {  
48         // fila ficou vazia  
49         q->last = NULL;  
50     }  
51  
52     free(temp);  
53     return q;  
54 }
```

Filas

○ REMOVE

```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

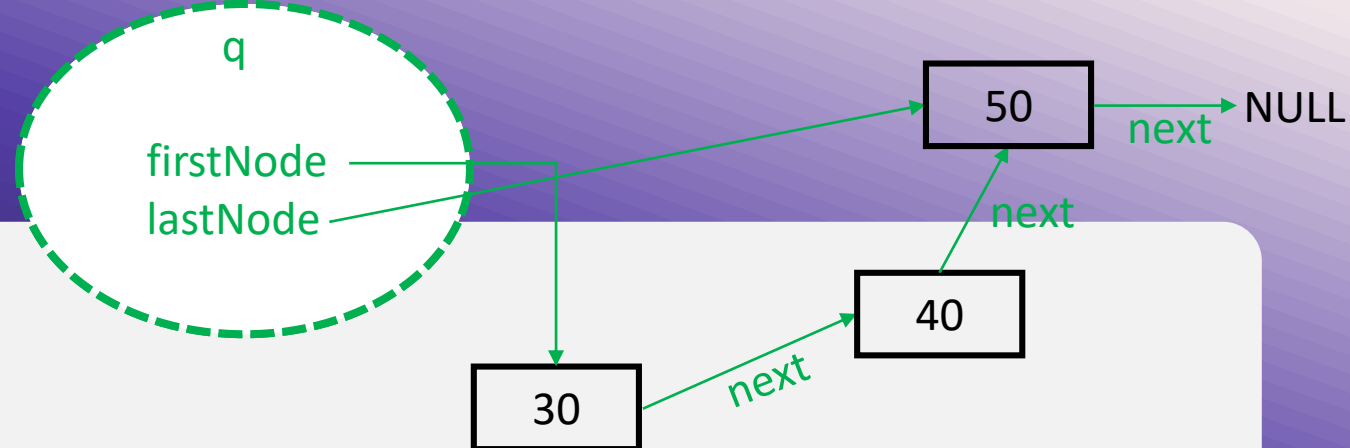


```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

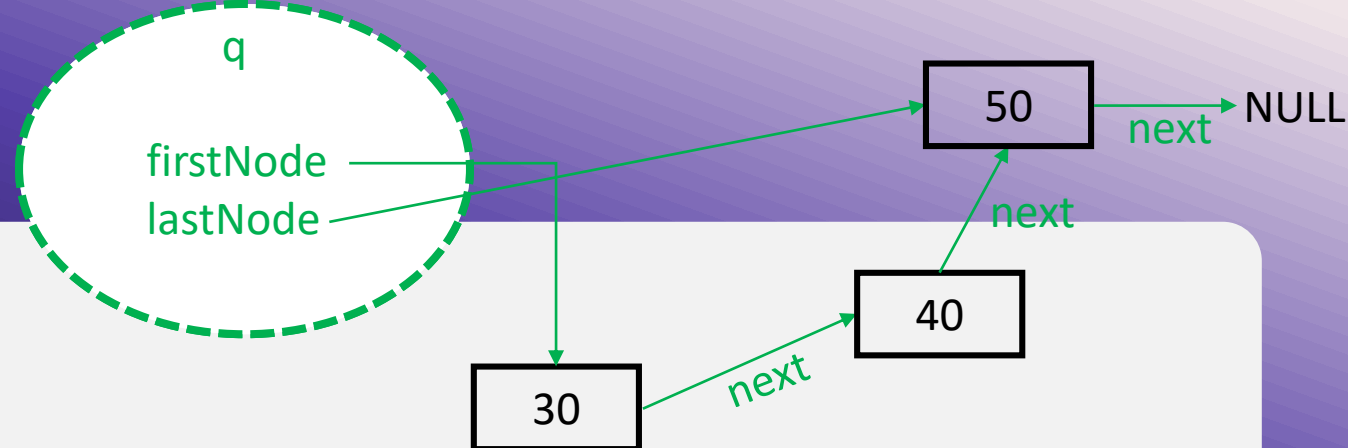


```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     q = remove_node(q);  
95     q = remove_node(q);  
96     q = remove_node(q);  
97  
98     print_queue(q);  
99  
100    free_queue(q);  
101    return 0;  
102 }
```

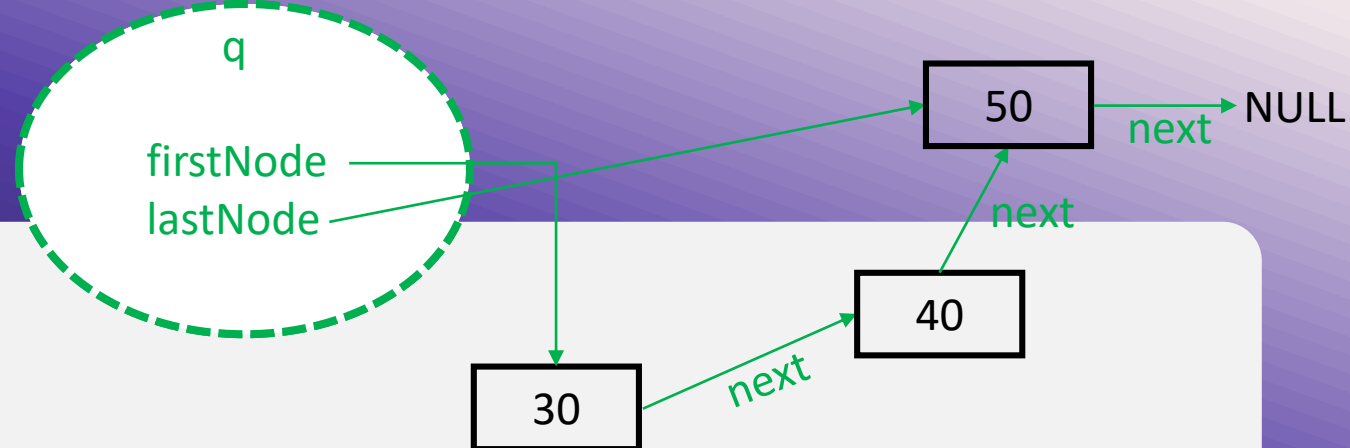


```
40 // Remove (dequeue) - remove do início da fila  
41 Queue* remove_node(Queue* q) {  
42     if (q->first == NULL) return q; // fila vazia  
43  
44     Node* temp = q->first;  
45     q->first = q->first->next;  
46  
47     if (q->first == NULL) {  
48         // fila ficou vazia  
49         q->last = NULL;  
50     }  
51  
52     free(temp);  
53     return q;  
54 }
```

Filas

○ REMOVE

```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

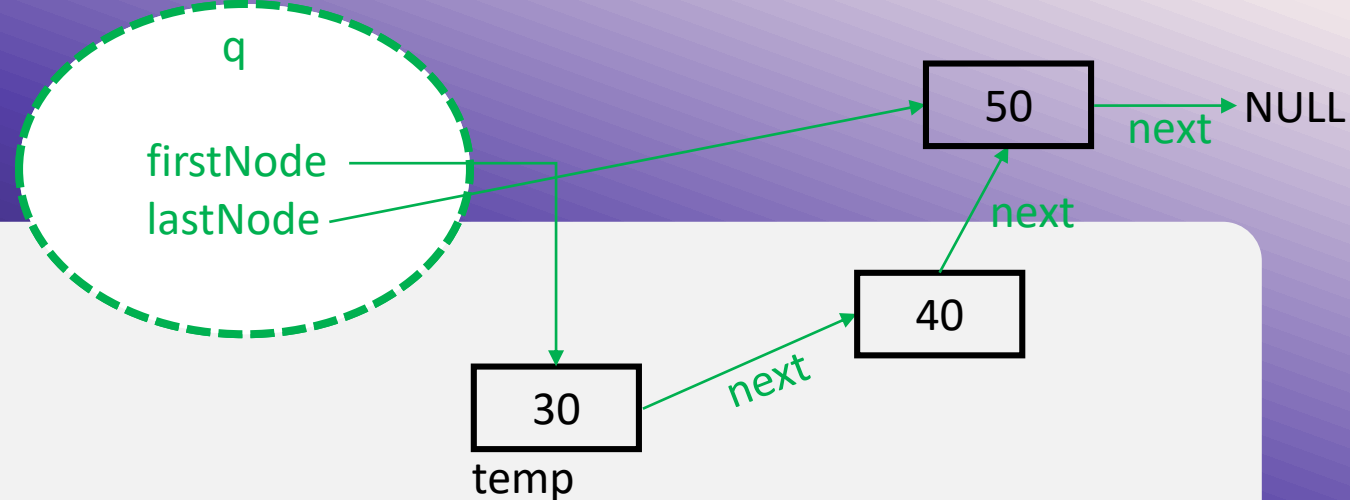


```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

```
83 int main() {  
84     Queue* q = (Queue*) malloc(sizeof(Queue));  
85     q->first = NULL;  
86     q->last = NULL;  
87  
88     q = insert_node(q, 10);  
89     q = insert_node(q, 20);  
90     q = insert_node(q, 30);  
91     q = insert_node(q, 40);  
92     q = insert_node(q, 50);  
93  
94     q = remove_node(q);  
95     q = remove_node(q);  
96     q = remove_node(q);  
97  
98     print_queue(q);  
99  
100    free_queue(q);  
101    return 0;  
102 }
```

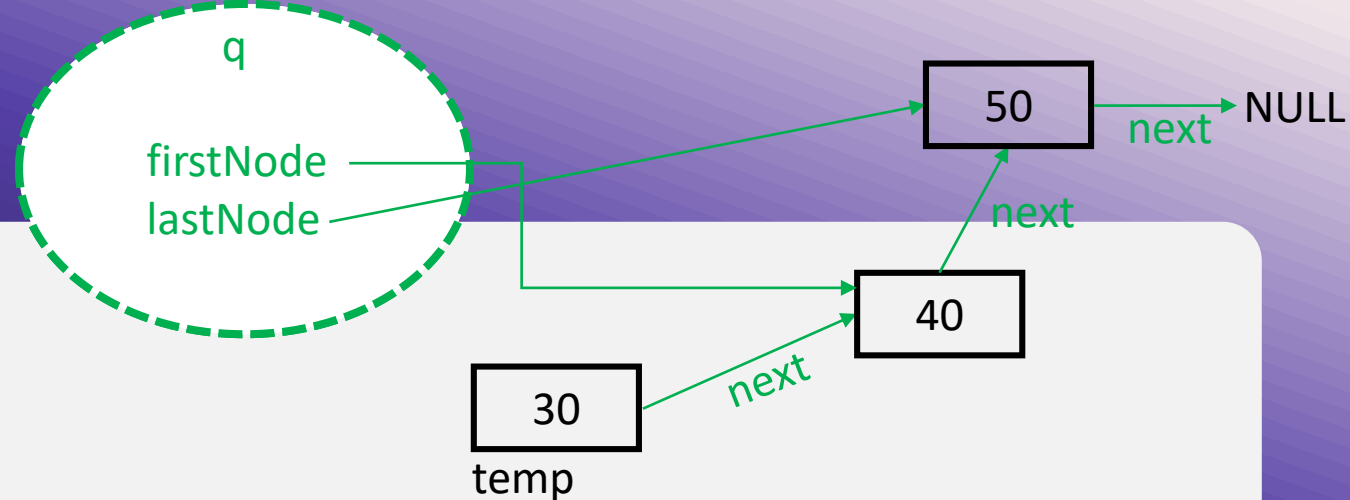


```
40 // Remove (dequeue) - remove do início da fila  
41 Queue* remove_node(Queue* q) {  
42     if (q->first == NULL) return q; // fila vazia  
43  
44     Node* temp = q->first;  
45     q->first = q->first->next;  
46  
47     if (q->first == NULL) {  
48         // fila ficou vazia  
49         q->last = NULL;  
50     }  
51  
52     free(temp);  
53     return q;  
54 }
```

Filas

○ REMOVE

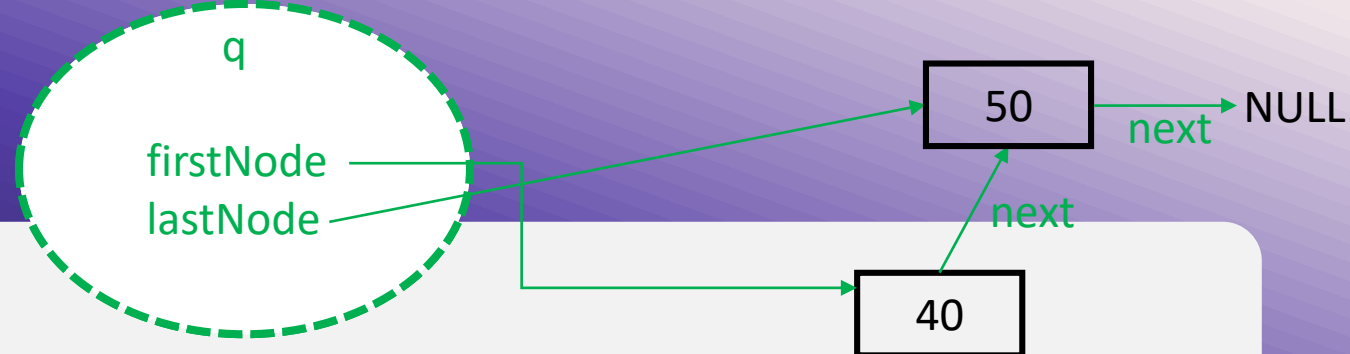
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```



```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

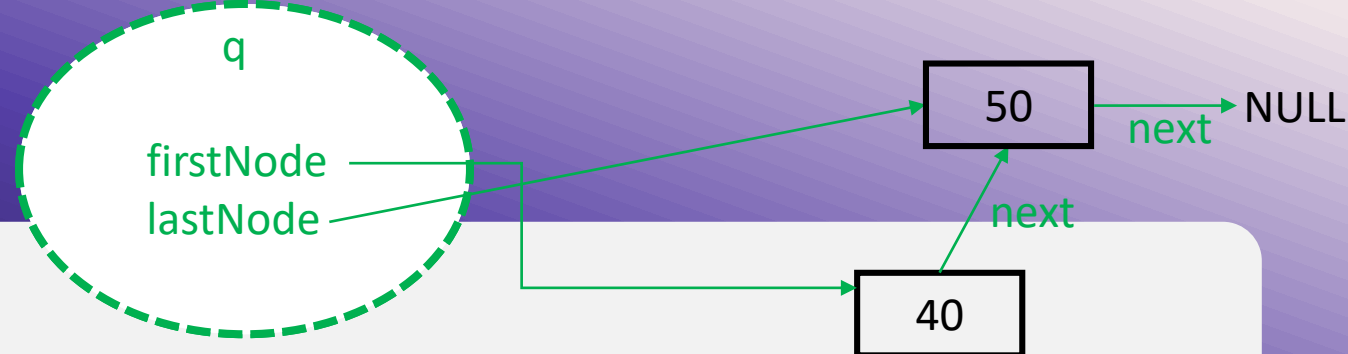


```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

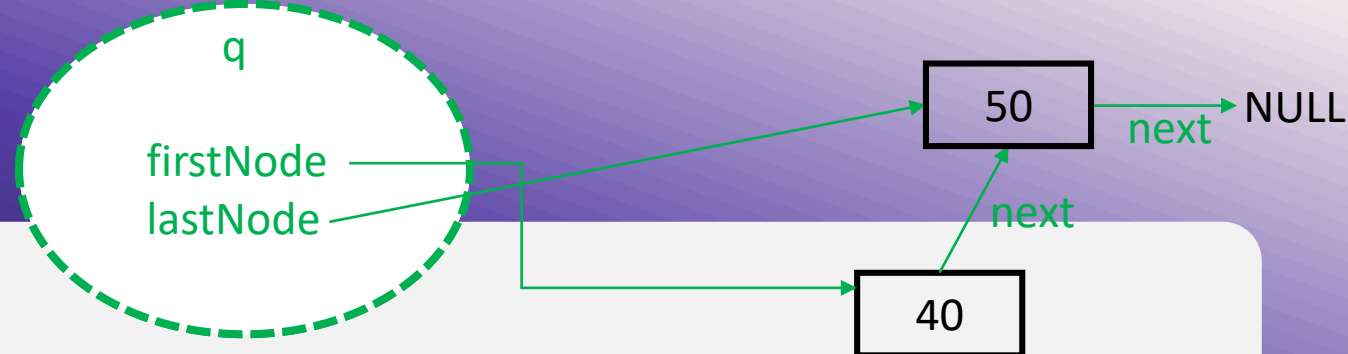


```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE



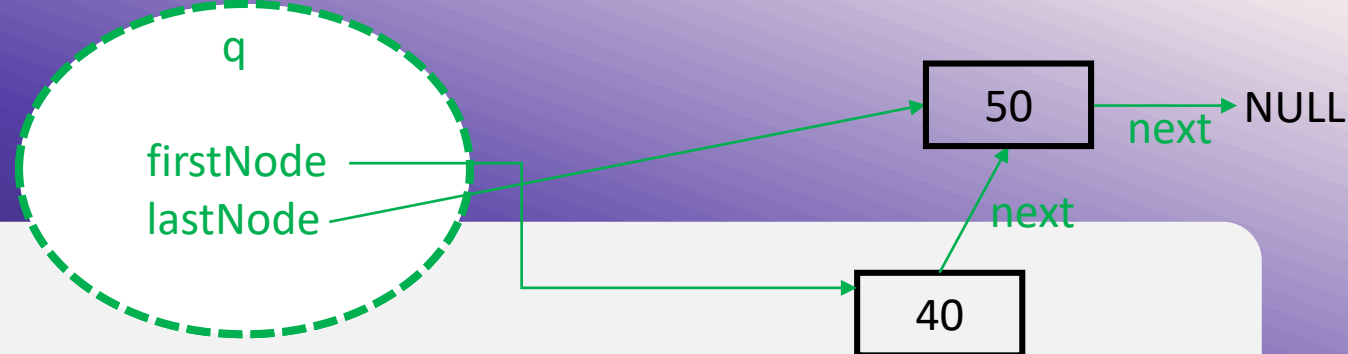
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas

○ REMOVE

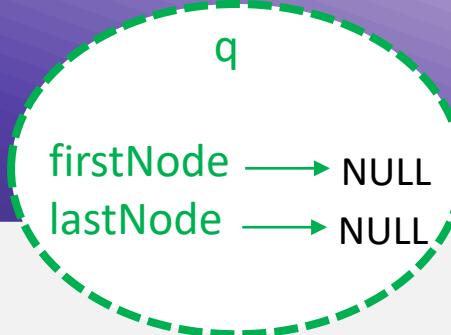
```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```



```
Queue: 40 -> 50 -> NULL
```

```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas



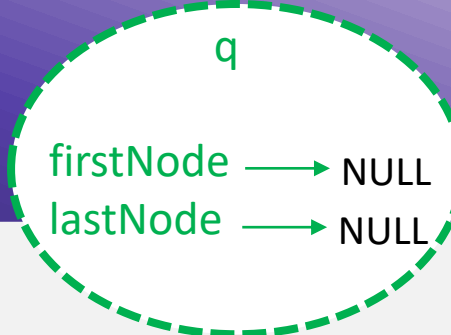
○ REMOVE

```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100     free_queue(q);
101     return 0;
102 }
```

```
Queue: 40 -> 50 -> NULL
```

```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```

Filas



○ REMOVE

```
83 int main() {
84     Queue* q = (Queue*) malloc(sizeof(Queue));
85     q->first = NULL;
86     q->last = NULL;
87
88     q = insert_node(q, 10);
89     q = insert_node(q, 20);
90     q = insert_node(q, 30);
91     q = insert_node(q, 40);
92     q = insert_node(q, 50);
93
94     q = remove_node(q);
95     q = remove_node(q);
96     q = remove_node(q);
97
98     print_queue(q);
99
100    free_queue(q);
101    return 0;
102 }
```

```
Queue: 40 -> 50 -> NULL
...Program finished with exit code 0
Press ENTER to exit console.
```

```
40 // Remove (dequeue) - remove do início da fila
41 Queue* remove_node(Queue* q) {
42     if (q->first == NULL) return q; // fila vazia
43
44     Node* temp = q->first;
45     q->first = q->first->next;
46
47     if (q->first == NULL) {
48         // fila ficou vazia
49         q->last = NULL;
50     }
51
52     free(temp);
53     return q;
54 }
```